Country & Weather Explorer

Technical documentation: project structure, features, methods and how they work

Platform	Flutter (Dart SDK ^3.13), tested on Android 16 (motorola edge 50 fusion)
State management	GetX 4.7 (controllers, Obx, bindings, routing)
Backend services	Firebase Authentication, REST Countries v5, Open-Meteo, OpenStreetMap tiles
Android package	country.weather
Source files	36 Dart files in lib/ + 1 unit-test file (about 3,000 lines)
Document date	10 October 2026

Contents

1. Overview 3

What the user can do . . . 3

Packages used . . . 3

2. Architecture 4

Dependency injection and routing . . . 4

Request flow (example: loading countries) . . . 4

Screen states . . . 4

3. Code directory 5

Every file and its job . . . 5

4. Features: methods and how they work 7

4.1 App start-up and session . . . 7

4.2 Login, sign-up and logout (Firebase Authentication) . . . 7

4.3 Country list (REST Countries v5) . . . 8

4.4 Search, region filter and favorites filter . . . 8

4.5 Favorites, recent searches and 'Continue exploring' (local storage) . . . 9

4.6 Country details . . . 9

4.7 Live weather (Open-Meteo) . . . 9

4.8 Map (flutter_map + OpenStreetMap) . . . 10

4.9 Error handling . . . 10

4.10 Keyboard and focus handling . . . 10

4.11 Theme and responsive layout . . . 11

5. External APIs 12

6. Testing and bug fixes 12

Manual test results . . . 12

Bugs found and fixed . . . 12

7. How to run 13

8. Full source code 14

1. Overview

Country & Weather Explorer is a Flutter app where a signed-in user can browse every country in the world, search and filter them, save favorites, and open a details page with live weather and an interactive map for that country.

What the user can do

- **Sign in / create an account** with email and password (Firebase Authentication). The session is remembered, so the next launch goes straight to the country list.
- **Browse 254 countries** with flag, name, capital and region, loaded from the REST Countries v5 API (paged, 100 per request). Pull down to refresh.
- **Search** by common or official name, **filter by region** (chips) and show **favorites only**.
- **Favorite** countries with the heart icon (saved on the device).
- **Recent searches** (last 5 countries opened from a search) and a **Continue exploring** banner for the last country viewed.
- **Country details**: official name, capital, region, population, currency, latitude/longitude, current **weather** (Open-Meteo) and a **map** with a full-screen interactive mode (OpenStreetMap).
- Friendly **loading, empty and error states** everywhere (no internet, timeout, server errors, missing data).

Packages used

Package	Used for	Where
get	State management (Rx/Obx), dependency injection, navigation, snackbars	Everywhere
firebase_core / firebase_auth	Firebase start-up, email/password sign-in & sign-up	main.dart, auth_service.dart
dio	HTTP client with timeouts	api_client.dart
connectivity_plus	Checks for a network before each request	api_client.dart
shared_preferences	Stores favorites, recent searches, last viewed country	storage_service.dart
flutter_map + latlong2	OpenStreetMap map with a marker	country_map.dart
cached_network_image	Downloads and caches flag images	flag_image.dart
flutter_lints / flutter_test	Static analysis rules and unit tests	dev only

2. Architecture

The code is split into layers. Each layer only talks to the one below it, which keeps screens free of HTTP and storage details:

Layer	Folder	Responsibility
UI (View)	lib/modules/*/..._view.dart, widgets/	Draws the screen. Reads state from its controller inside Obx(...) so it rebuilds automatically. No business logic.
Controller	lib/modules/*/..._controller.dart	Holds screen state as Rx values (.obs), handles user actions, calls repositories and services.
Repository	lib/data/repositories/	Builds API requests (URL, query, headers) and turns JSON into model objects.
Service	lib/data/services/	Long-lived app services: AuthService (Firebase) and StorageService (SharedPreferences).
Model	lib/data/models/	Plain Dart classes (Country, Currency, Weather) with defensive fromJson parsing.
Core	lib/core/	Shared building blocks: ApiClient (Dio), AppException, config, theme, formatters, validators, ViewStatus.

Dependency injection and routing

At start-up `initDependencies()` registers the app-wide singletons once with `Get.put(..., permanent: true)`: StorageService, AuthService, ApiClient, CountryRepository and WeatherRepository. Screen controllers are *not* global: each `GetPage` in `app_pages.dart` has a binding that `Get.lazyPut` its controller, so the controller is created when the page opens and disposed (`onClose()`) when the page is removed.

Request flow (example: loading countries)

Step	Code	What happens
1	HomeController.onInit()	Calls fetchCountries() when the home page opens.
2	CountryRepository.getCountries()	Checks the API key, then asks for page after page (limit 100, offset 0/100/200...) until meta.more is false.
3	ApiClient.get()	Checks connectivity, sends the GET with Dio, converts any DioException into an AppException with a user-friendly message.
4	Country.fromJson()	Turns each JSON object into a Country (missing fields become 'N/A' instead of crashing).
5	HomeController	Stores the list in countries (RxList), sets status to success/empty/error and runs _applyFilters().
6	HomeView (Obx)	Rebuilds automatically and shows the skeleton, list, empty or error view based on status.

Screen states

Every loading section uses the `ViewStatus` enum: `loading`, `success`, `empty` and `error`. The view switches on it, so each state always has a clear UI (skeleton cards, list, message, or error with a retry button via `StatusView`).

3. Code directory

Folder tree of the Dart source. Platform folders (android/, ios/, web/, windows/, macos/, linux/) are the standard Flutter shells; the only custom Android files are `app/build.gradle.kts` (package `country.weather`) and `app/google-services.json`.

```
country_wearther_task/
|-- pubspec.yaml                packages and SDK version
|-- README.md                   setup guide
|-- android/app/
|   |-- build.gradle.kts        applicationId = country.weather, Google services plugin
|   |-- google-services.json    Firebase config for Android
|   `-- src/main/kotlin/country/weather/MainActivity.kt
|-- test/
|   `-- models_test.dart
`-- lib/
    |-- main.dart
    |-- firebase_options.dart
    |-- core/
    |   |-- dependencies.dart
    |   |-- config/      app_config.dart
    |   |-- network/     api_client.dart, app_exception.dart
    |   |-- theme/       app_colors.dart, app_theme.dart
    |   `-- utils/       formatters.dart, validators.dart, view_status.dart
    |-- data/
    |   |-- models/        country_model.dart, weather_model.dart
    |   |-- repositories/  country_repository.dart, weather_repository.dart
    |   `-- services/      auth_service.dart, storage_service.dart
    |-- modules/
    |   |-- login/    login_controller.dart, login_view.dart
    |   |-- home/     home_controller.dart, home_view.dart
    |   |   `-- widgets/  country_card, country_list_skeleton,
    |   |                 last_viewed_card, recent_searches
    |   `-- details/  country_details_controller.dart, country_details_view.dart
    |       `-- widgets/  info_tile, map_section, weather_section
    |-- routes/      app_routes.dart, app_pages.dart
    `-- widgets/     country_map, flag_image, section_card, status_view
```

Every file and its job

File	Lines	Purpose
`lib/main.dart`	66	App entry point: initialises Firebase and dependencies, picks the first route (home if signed in, else login).
`lib/firebase_options.dart`	17	Placeholder for flutterfire configure. Not used on Android (Firebase reads android/app/google-services.json).
`lib/core/dependencies.dart`	18	initDependencies(): registers services and repositories.
`lib/core/config/app_config.dart`	36	API URLs, API key, page size, timeouts, constants.
`lib/core/network/api_client.dart`	76	Dio wrapper: connectivity check + error mapping.
`lib/core/network/app_exception.dart`	24	AppException and AppErrorType (friendly errors).
`lib/core/theme/app_colors.dart`	33	Colour palette and gradients.
`lib/core/theme/app_theme.dart`	107	AppSpacing constants and the Material 3 ThemeData.
`lib/core/utils/formatters.dart`	33	Number and coordinate formatting helpers.
`lib/core/utils/validators.dart`	19	Email and password form validators.
`lib/core/utils/view_status.dart`	2	ViewStatus enum (loading/success/empty/error).
`lib/data/models/country_model.dart`	98	Country and Currency models + fromJson.

File	Lines	Purpose
lib/data/models/weather_model.dart	82	Weather model, fromJson, WMO code to text.
lib/data/repositories/country_repository.dart	52	getCountries(): paged REST Countries request.
lib/data/repositories/weather_repository.dart	24	getCurrentWeather(lat, lng): Open-Meteo request.
lib/data/services/auth_service.dart	58	Firebase sign-in/up/out + error messages.
lib/data/services/storage_service.dart	61	Favorites, recent searches, last viewed (SharedPreferences).
lib/modules/login/login_controller.dart	61	Login form state and submit().
lib/modules/login/login_view.dart	202	Login / sign-up screen UI.
lib/modules/home/home_controller.dart	170	Country list state, search, filters, navigation, logout.
lib/modules/home/home_view.dart	272	Home screen: search bar, chips, list, states.
lib/modules/home/widgets/country_card.dart	115	One country row + RegionBadge.
lib/modules/home/widgets/country_list_skeleton.dart	51	Grey placeholder cards while loading.
lib/modules/home/widgets/last_viewed_card.dart	66	'Continue exploring' banner.
lib/modules/home/widgets/recent_searches.dart	58	Recent search chips + Clear.
lib/modules/details/country_details_controller.dart	59	Selected country, weather state, favorite toggle.
lib/modules/details/country_details_view.dart	215	Details screen layout (phone/tablet).
lib/modules/details/widgets/info_tile.dart	65	Icon + label + value fact tile.
lib/modules/details/widgets/map_section.dart	80	Map preview + FullScreenMap page.
lib/modules/details/widgets/weather_section.dart	230	Weather card with its own loading/error/empty states.
lib/routes/app_routes.dart	7	Route name constants.
lib/routes/app_pages.dart	33	GetPage list with per-page controller bindings.
lib/widgets/country_map.dart	59	Reusable FlutterMap with OSM tiles and a marker.
lib/widgets/flag_image.dart	53	Cached flag image with emoji fallback.
lib/widgets/section_card.dart	50	White card with icon + title header.
lib/widgets/status_view.dart	78	Centred empty/error message with action button.
test/models_test.dart	112	Unit tests: models, validators, formatters (9 tests).

4. Features: methods and how they work

For each feature: the files involved, the important methods, and the step-by-step flow when the user uses it.

4.1 App start-up and session

Files: `main.dart`, `core/dependencies.dart`, `routes/app_pages.dart`

Method / member	What it does
`main()`	Entry point. Initialises Flutter bindings, Firebase and dependencies, then runApp.
`Firebase.initializeApp()`	Connects to Firebase using google-services.json. If it fails, _StartupErrorApp shows a 'Setup required' screen instead of crashing.
`initDependencies()`	Registers StorageService (awaits SharedPreferences), AuthService, ApiClient and both repositories as permanent.
`CountryWeatherApp.build()`	Creates GetMaterialApp with the theme and routes. initialRoute = /home if AuthService.isLoggedIn, else /login.
`AppPages.pages`	Maps /login, /home, /details to views and lazily-created controllers.

How it works

1. Android starts MainActivity (package country.weather), which runs main().
2. Firebase starts. Firebase Auth restores the saved session from disk automatically.
3. Services and repositories are registered in GetX so any controller can Get.find() them.
4. If FirebaseAuth.currentUser is not null the user lands on Home, otherwise on Login.

4.2 Login, sign-up and logout (Firebase Authentication)

Files: `modules/login/login_controller.dart`, `modules/login/login_view.dart`, `data/services/auth_service.dart`, `core/utils/validators.dart`

Method / member	What it does
`LoginController.submit()`	Ignores taps while a request runs, validates the form, hides the keyboard, calls signIn or signUp, then Get.offAllNamed('/home').
`LoginController.toggleMode()`	Switches between 'Sign in' and 'Create account' and clears the error.
`togglePasswordVisibility()`	Shows/hides the password (obscurePassword.obs).
`AuthService.signIn(email, pw)`	signInWithEmailAndPassword; FirebaseAuthException becomes AppException.
`AuthService.signUp(email, pw)`	createUserWithEmailAndPassword (same error handling).
`AuthService._messageFor(code)`	Maps Firebase codes (invalid-credential, email-already-in-use, network-request-failed, ...) to readable messages.
`Validators.email / .password`	Empty check, email regex, minimum 6 characters.
`HomeController.logout()`	AuthService.signOut() then Get.offAllNamed('/login'), which also disposes the home controller.

How it works

1. The Form re-validates each field while the user types (AutovalidateMode.onUserInteraction), so errors disappear as soon as a field is fixed.
2. On Sign in, submit() validates; if valid it shows a spinner on the button (isLoading) and calls Firebase.
3. Success: all previous routes are removed and Home opens. Failure: the friendly message is shown in a red banner (_ErrorBanner).
4. Logout from Home asks for confirmation in an AlertDialog first.

4.3 Country list (REST Countries v5)

Files: `data/repositories/country_repository.dart`, `core/network/api_client.dart`, `data/models/country_model.dart`, `modules/home/home_controller.dart`, `modules/home/home_view.dart`

Method / member	What it does
`CountryRepository.getCountries()`	Throws a clear error if the API key is empty. Loops pages: ?response_fields=...&limit=100&offset=N with header Authorization: Bearer . Stops when data.meta.more is false (max 10 pages). Sorts by name.
`ApiClient.get(url, query, headers)`	Checks connectivity, sends the request, returns response.data or throws AppException.
`ApiClient._mapDioError(e)`	Timeout -> timeout message; connection error -> no internet; 401/403 -> check API key; 429 -> too many requests; other -> server error (code).
`Country.fromJson(json)`	Reads names, codes.alpha_3, capitals, flag, region, population, currencies, coordinates. Falls back to the capital's coordinates; missing values become 'N/A'.
`HomeController.fetchCountries()`	Shows the skeleton, loads the list, sets status, applies filters.
`HomeController.refreshCountries()`	Pull-to-refresh: reloads without the skeleton. If it fails while a list is shown, the list stays and a snackbar explains the error.
`CountryCard`	Flag (Hero animation to details), name, capital, RegionBadge and a heart button.

How it works

1. Only the fields the app needs are requested (response_fields) to keep responses small; 254 countries arrive in 3 requests.
2. The list is a SliverGrid with maxCrossAxisExtent 520: one column on phones, two or more on tablets. Each card is 84 px high.
3. The list is wrapped in SliverSafeArea so the last card is never hidden behind the Android navigation bar.

4.4 Search, region filter and favorites filter

Files: `modules/home/home_controller.dart`, `modules/home/home_view.dart`

Method / member	What it does
`onSearchChanged(value)`	Updates searchQuery (RxString) on every keystroke.
`debounce(searchQuery, ..., 300 ms)`	Runs the filter only after typing pauses for 300 ms, so it doesn't re-filter on every letter.
`_applyFilters()`	Keeps countries whose common or official name contains the query AND whose region matches the chip AND (if favorites-only) that are favorites. Result goes into visibleCountries.
`_applyFiltersAndScrollToTop()`	Same, then scrolls the list to the top so new results start at the beginning.
`selectRegion(region) / toggleFavoritesOnly()`	Change the filter and close the keyboard.
`regions (getter)`	Builds the chip list from the regions in the data: All, Africa, Americas, Antarctic, Asia, Europe, Oceania.
`clearSearch()`	Empties the text field and query and refreshes the list.

How it works

1. ever(selectedRegion) / ever(showFavoritesOnly) re-filter whenever a chip changes.
2. ever(storage.favorites) re-filters (without scrolling) when a heart is tapped, so the favorites-only list updates live.

3. Each empty result has its own message: 'No results' for search, 'No favorites yet' and 'Nothing here' for a region.

4.5 Favorites, recent searches and 'Continue exploring' (local storage)

Files: `data/services/storage_service.dart`, `modules/home/widgets/recent_searches.dart`, `modules/home/widgets/last_viewed_card.dart`

Method / member	What it does
`StorageService.init()`	Loads the three saved values from SharedPreferences into reactive lists at start-up.
`toggleFavorite(code)`	Adds/removes a country code (e.g. IND) and saves the list.
`addRecentSearch(code)`	Moves the code to the front, keeps at most 5, saves.
`clearRecentSearches()`	Empties the list (the 'Clear' button).
`saveLastSelected(code)`	Called when a details page opens; powers the 'Continue exploring' banner.
`HomeController.openCountry(country)`	Adds a recent search only if the user was searching, closes the keyboard, then opens /details with the country as argument.
`recentCountries / lastSelectedCountry`	Turn saved codes back into Country objects from the loaded list.

How it works

1. Only country codes are stored, never full objects, so the data stays small and is always matched against fresh API data.
2. Because favorites/recentSearches/lastSelected are Rx values, the home screen and the details heart update instantly on every screen.
3. Recent searches and the banner are hidden while searching or in favorites-only mode.

4.6 Country details

Files: `modules/details/country_details_controller.dart`, `modules/details/country_details_view.dart`, `modules/details/widgets/info_tile.dart`, `core/utils/formatters.dart`

Method / member	What it does
`CountryDetailsController.country`	The Country passed by Get.toNamed(arguments: country). If missing, a 'Country not found' view is shown.
`onInit()`	Saves the country as last viewed and starts fetchWeather().
`toggleFavorite() / isFavorite`	Heart in the app bar; same storage as Home.
`_buildAppBar()`	SliverAppBar 230 px tall with the flag (Hero) and a dark gradient so the title is readable.
`_buildFacts()`	Six InfoTiles in a Wrap: capital, region, population, currency, latitude, longitude.
`InfoTile(singleLine: true)`	Numbers (population, coordinates) stay on one line and shrink to fit instead of breaking mid-number.
`Formatters.groupedNumber / latitude / longitude`	1429404000 -> 1,429,404,000; 20.0 -> 20.0000° N.

How it works

1. LayoutBuilder: width >= 800 shows facts and weather/map side by side; phones show one column.
2. The scroll content is inside SliverSafeArea so the map card is fully visible above the navigation bar.

4.7 Live weather (Open-Meteo)

Files: `data/repositories/weather_repository.dart`, `data/models/weather_model.dart`, `modules/details/widgets/weather_section.dart`

Method / member	What it does
`WeatherRepository.getCurrentWeather(lat, lng)`	GET api.open-meteo.com/v1/forecast?latitude&longitude&current=temperature_2m,apparent_temperature,relative_humidity_2m,wind_speed_10m,weather_code,is_day. No API key.
`Weather.fromJson(json)`	Reads the 'current' block; throws invalidResponse if temperature is missing.
`Weather.condition`	Converts the WMO weather code to text (Clear sky, Rain, Snow, ...).
`fetchWeather()`	Sets weatherStatus: empty if no coordinates, then loading -> success or error.
`WeatherSection`	AnimatedSwitcher between loading, error (with Retry), empty and the gradient weather card.
`_iconFor(weather)`	Picks a sun/moon/cloud/rain/snow/storm icon from the code and day/night.

How it works

1. Weather has its own status, separate from the page, so if it fails the rest of the details screen still works and only the weather card shows Retry.

4.8 Map (flutter_map + OpenStreetMap)

Files: `widgets/country_map.dart`, `modules/details/widgets/map_section.dart`

Method / member	What it does
`CountryMap(lat, lng, zoom, interactive)`	FlutterMap with an OSM TileLayer, a red marker and the required attribution.
`MapSection`	240 px non-interactive preview with an Expand button; 'Location not available' if there are no coordinates.
`FullScreenMap`	Pushed with Get.to(); interactive (drag, pinch zoom; rotation off), zoom 5.
`AppConfig.mapUserAgent`	'country.weather', sent with tile requests as OSM's tile policy requires.

How it works

1. The preview disables gestures so scrolling the details page isn't captured by the map.

4.9 Error handling

Files: `core/network/app_exception.dart`, `core/network/api_client.dart`, `widgets/status_view.dart`

Method / member	What it does
`AppException`	Exception with a user-facing message and a type (noInternet, timeout, server, invalidResponse, unknown).
`ApiClient._ensureConnected()`	connectivity_plus check; throws noInternet before even sending.
`StatusView / StatusView.error`	Reusable centred icon + title + message + action button.
`_StartupErrorApp`	Shown if Firebase fails to start.

How it works

1. Controllers catch AppException and show e.message; any other error falls back to a generic message. Nothing crashes the app.

4.10 Keyboard and focus handling

Files: `modules/home/home_controller.dart`, `modules/home/home_view.dart`

Method / member	What it does
`searchFocusNode`	FocusNode owned by the controller and attached to the search field.
`hideKeyboard()`	searchFocusNode.unfocus(): closes the keyboard but keeps the text.
`TextField.onTapOutside / onSubmitted`	Close the keyboard when tapping elsewhere or pressing Search.
`keyboardDismissBehavior.onDrag`	Scrolling the list closes the keyboard.

How it works

1. openCountry() unfocuses before navigating; otherwise Flutter restores focus to the search field when coming back and the keyboard pops up again.

4.11 Theme and responsive layout

Files: `core/theme/app_theme.dart`, `core/theme/app_colors.dart`

Method / member	What it does
`AppTheme.light`	Material 3 theme from the indigo seed colour: inputs, buttons, chips, app bar, snackbars.
`AppSpacing`	xs 4, sm 8, md 16, lg 24, xl 32, radius 16 - used everywhere for consistent spacing.
`AnnotatedRegion<SystemUiOverlayStyle>`	Login screen: white status-bar icons on the dark gradient.

How it works

1. Layouts adapt by width (SliverGrid on Home, LayoutBuilder on Details), and the login card is limited to 440 px wide on large screens.

5. External APIs

Service	Endpoint	Auth	Used by
REST Countries v5	GET https://api.restcountries.com/countries/v5?response_fields=...&limit=100&offset=N	Authorization: Bearer <key> (free key from restcountries.com/sign-up)	CountryRepository
Open-Meteo	GET https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&current=...	None	WeatherRepository
OpenStreetMap tiles	https://tile.openstreetmap.org/{z}/{x}/{y}.png	None (user agent required)	CountryMap
Firebase Auth	Firebase SDK (email/password)	google-services.json	AuthService

API key. The key is read with String.fromEnvironment('COUNTRIES_API_KEY', defaultValue: ...). It can be passed at build time with `--dart-define=COUNTRIES_API_KEY=your_key`. The real key is masked in the code listing of this document.

6. Testing and bug fixes

The app was tested end to end on a real device (motorola edge 50 fusion, Android 16) and with the unit tests (`flutter test`: 9 tests pass, `flutter analyze`: no issues).

Manual test results

Area	What was checked	Result
Build & launch	Debug APK builds, installs, starts without crashing	**Fixed, then pass**
Login	Empty fields, invalid email, short password, wrong password, sign-up toggle	**Fixed, then pass**
Session / logout	Stays signed in after restart; logout dialog Cancel and Log out	**Pass**
Country list	254 countries load with flags, capitals, regions	**Fixed, then pass**
Search	Results, 'No results' state, clear button	**Pass**
Filters	Region chips, favorites-only, list jumps to top on change	**Fixed, then pass**
Favorites	Heart on card and details; count chip updates; saved	**Pass**
Recent searches	Chip shows country name and opens details	**Fixed, then pass**
Continue exploring	Banner shows last viewed country	**Pass**
Details	Facts, population formatting, layout above nav bar	**Fixed, then pass**
Weather	Temperature, condition, humidity, wind load	**Pass**
Map	Preview, Expand to full screen, pan	**Pass**
Keyboard	Doesn't reopen after returning from details; closes on scroll/tap	**Fixed, then pass**

Not tested on the device: no-internet mode (the phone was connected to the PC over Wi-Fi, so turning Wi-Fi off would have cut the connection), tablet layout and iOS. The error-handling code for these paths was reviewed instead.

Bugs found and fixed

#	Bug	Cause	Fix (file)
1	Build failed: 'No matching client for package name'	MainActivity was still in the old com.example package while Firebase expects country.weather	Moved MainActivity.kt to kotlin/country/weather/

#	Bug	Cause	Fix (file)
2	Countries never loaded	API key placed as the variable *name* in String.fromEnvironment, so the key was empty	Key moved to defaultValue (app_config.dart)
3	Wrong API URL	?q=canada appended to the base URL	Restored base URL (app_config.dart)
4	'Bottom overflowed by 7 px' on every card	Fixed card height 72 px too small	mainAxisExtent 84 (home_view.dart)
5	Keyboard reopened after returning from a country	Search field kept focus during navigation	FocusNode + hideKeyboard() (home_controller/view)
6	Recent-search chip showed only the flag	Chip theme label style had no colour	Label colour set (app_theme.dart, recent_searches.dart)
7	Last card and map hidden behind the navigation bar	Android 15+ edge-to-edge with no bottom inset	SliverSafeArea (home_view, country_details_view)
8	Population split as '1,429,404,00 / 0'	Long number wrapped in a narrow tile	InfoTile singleLine + FittedBox (info_tile.dart)
9	Changing a filter kept the old scroll position	No scroll reset	ScrollController.jumpTo(0) on filter change (home_controller)
10	A failed refresh replaced the list with an error	Error state always used	Keep list + snackbar (home_controller._onFetchError)
11	Validation error stayed after fixing the field	Errors only re-checked on submit	AutovalidateMode.onUserInteraction (login_view.dart)
12	Dark status-bar icons on the purple login screen	No AppBar to set icon brightness	AnnotatedRegion light style (login_view.dart)
13	Pressing Done twice could send two logins	No guard while loading	Return early if isLoading (login_controller.dart)
14	Map tiles sent the old package name	Stale user agent	mapUserAgent = 'country.weather' (app_config.dart)

7. How to run

```
flutter pub get
flutter run                                  # uses the key in app_config.dart
flutter run --dart-define=COUNTRIES_API_KEY=rc_live_xxx   # or pass a key
flutter test                                 # unit tests
flutter analyze                              # static analysis
flutter build apk --release                  # release APK
```

- Firebase: the project must have **Authentication** → **Email/Password** enabled. Android reads `android/app/google-services.json`.
- Build warning: firebase_core and firebase_auth still apply the Kotlin Gradle Plugin. It builds today, but a future Flutter version will need updated plugin versions.

8. Full source code

Every Dart file in lib/ plus the unit tests, with line numbers, in the same order as the directory table. Characters the code font can't draw (emoji flags) are written as \u{...} escapes.

lib/main.dart 66 lines

```dart
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'core/config/app_config.dart';
import 'core/dependencies.dart';
import 'core/theme/app_theme.dart';
import 'data/services/auth_service.dart';
import 'routes/app_pages.dart';
import 'routes/app_routes.dart';
import 'widgets/status_view.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  try {
    await Firebase.initializeApp();
  } catch (e) {
    // Show a friendly screen instead of crashing if Firebase isn't set up.
    runApp(_StartupErrorApp(message: e.toString()));
    return;
  }

  await initDependencies();
  runApp(const CountryWeatherApp());
}

class CountryWeatherApp extends StatelessWidget {
  const CountryWeatherApp({super.key});

  @override
  Widget build(BuildContext context) {
    // Firebase restores the previous session, so logged-in users skip the login screen.
    final isLoggedIn = Get.find<AuthService>().isLoggedIn;

    return GetMaterialApp(
      title: AppConfig.appName,
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      initialRoute: isLoggedIn ? AppRoutes.home : AppRoutes.login,
      getPages: AppPages.pages,
      defaultTransition: Transition.cupertino,
    );
  }
}

class _StartupErrorApp extends StatelessWidget {
  final String message;

  const _StartupErrorApp({required this.message});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      home: Scaffold(
        body: StatusView(
          icon: Icons.settings_suggest_rounded,
          title: 'Setup required',
          message: message,
        ),
      ),
    );
  }
}
```

lib/firebase_options.dart 17 lines

```dart
// PLACEHOLDER - replace this file by running:
//
//   dart pub global activate flutterfire_cli
//   flutterfire configure
//
// FlutterFire CLI will overwrite this file with your Firebase project's keys.

import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;

class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    throw UnsupportedError(
      'Firebase is not configured yet. Run "flutterfire configure" '
      'to generate lib/firebase_options.dart.',
    );
  }
}
```

lib/core/dependencies.dart 18 lines

```dart
import 'package:get/get.dart';

import '../data/repositories/country_repository.dart';
import '../data/repositories/weather_repository.dart';
import '../data/services/auth_service.dart';
import '../data/services/storage_service.dart';
import 'network/api_client.dart';

/// Registers app-wide services and repositories once at startup.
/// Screen controllers are registered per page in `AppPages`.
Future<void> initDependencies() async {
  await Get.putAsync(() => StorageService().init(), permanent: true);
  Get.put(AuthService(), permanent: true);

  final apiClient = Get.put(ApiClient(), permanent: true);
  Get.put(CountryRepository(apiClient), permanent: true);
  Get.put(WeatherRepository(apiClient), permanent: true);
}
```

lib/core/config/app_config.dart 36 lines

```dart
/// Central place for API URLs, keys and app-wide constants.
///
/// The REST Countries key can be passed at build time:
/// `flutter run --dart-define=COUNTRIES_API_KEY=your_key`
/// or pasted into [countriesApiKey]'s defaultValue.
class AppConfig {
  AppConfig._();

  static const String appName = 'Country & Weather Explorer';

  // REST Countries (https://restcountries.com) - v5 requires an API key.
  static const String countriesBaseUrl = 'https://api.restcountries.com/countries/v5';
  static const String countriesApiKey = String.fromEnvironment(
    'COUNTRIES_API_KEY',
    defaultValue: 'rc_live_********  (masked)',
  );

  /// Only the fields the app actually uses, to keep responses small.
  static const String countriesFields =
      'names.common,names.official,codes.alpha_3,capitals,flag.url_png,'
      'flag.emoji,region,subregion,population,currencies,coordinates';

  /// Max page size allowed on the free plan.
  static const int countriesPageSize = 100;

  // Open-Meteo (https://open-meteo.com) - free, no API key required.
  static const String weatherBaseUrl = 'https://api.open-meteo.com/v1/forecast';

  // OpenStreetMap tiles used by flutter_map.
  static const String mapTileUrl = 'https://tile.openstreetmap.org/{z}/{x}/{y}.png';
  // Must match the real applicationId - OSM's tile policy requires an identifying user agent.
  static const String mapUserAgent = 'country.weather';

  static const Duration requestTimeout = Duration(seconds: 15);
  static const int maxRecentSearches = 5;
}
```

lib/core/network/api_client.dart 76 lines

```dart
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:dio/dio.dart';

import '../config/app_config.dart';
import 'app_exception.dart';

/// Thin wrapper around Dio that turns every failure into an [AppException].
class ApiClient {
  final Dio _dio;
  final Connectivity _connectivity;

  ApiClient({Dio? dio, Connectivity? connectivity})
      : _dio = dio ??
            Dio(BaseOptions(
              connectTimeout: AppConfig.requestTimeout,
              receiveTimeout: AppConfig.requestTimeout,
            )),
        _connectivity = connectivity ?? Connectivity();

  Future<dynamic> get(
    String url, {
    Map<String, dynamic>? query,
    Map<String, String>? headers,
  }) async {
    await _ensureConnected();
    try {
      final response = await _dio.get(
        url,
        queryParameters: query,
        options: Options(headers: headers),
      );
      if (response.data == null) throw const AppException.invalidResponse();
      return response.data;
    } on DioException catch (e) {
      throw _mapDioError(e);
    }
  }

  Future<void> _ensureConnected() async {
    final results = await _connectivity.checkConnectivity();
    if (results.every((r) => r == ConnectivityResult.none)) {
      throw const AppException.noInternet();
    }
  }

  AppException _mapDioError(DioException e) {
    switch (e.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.sendTimeout:
      case DioExceptionType.receiveTimeout:
        return const AppException.timeout();
      case DioExceptionType.connectionError:
        return const AppException.noInternet();
      case DioExceptionType.badResponse:
        final code = e.response?.statusCode ?? 0;
        if (code == 401 || code == 403) {
          return const AppException(
            'Access denied by the server. Please check the API key.',
            type: AppErrorType.server,
          );
        }
        if (code == 429) {
          return const AppException(
            'Too many requests. Please wait a moment and try again.',
            type: AppErrorType.server,
          );
        }
        return AppException(
          'Server error ($code). Please try again later.',
          type: AppErrorType.server,
        );
      default:
        return const AppException('Something went wrong. Please try again.');
    }
  }
}
```

lib/core/network/app_exception.dart 24 lines

```dart
/// A user-friendly error that the UI can show directly.
class AppException implements Exception {
  final String message;
  final AppErrorType type;

  const AppException(this.message, {this.type = AppErrorType.unknown});

  const AppException.noInternet()
      : message = 'No internet connection. Please check your network and try again.',
        type = AppErrorType.noInternet;

  const AppException.timeout()
      : message = 'The request timed out. Please try again.',
        type = AppErrorType.timeout;

  const AppException.invalidResponse()
      : message = 'We received an unexpected response from the server.',
        type = AppErrorType.invalidResponse;

  @override
  String toString() => message;
}

enum AppErrorType { noInternet, timeout, server, invalidResponse, unknown }
```

lib/core/theme/app_colors.dart 33 lines

```dart
import 'package:flutter/material.dart';

class AppColors {
  AppColors._();

  static const Color primary = Color(0xFF4F46E5);
  static const Color primaryDark = Color(0xFF3730A3);
  static const Color accent = Color(0xFF0EA5E9);

  static const Color background = Color(0xFFF5F6FA);
  static const Color surface = Colors.white;
  static const Color border = Color(0xFFE5E7EB);

  static const Color textPrimary = Color(0xFF111827);
  static const Color textSecondary = Color(0xFF6B7280);

  static const Color success = Color(0xFF10B981);
  static const Color warning = Color(0xFFF59E0B);
  static const Color error = Color(0xFFEF4444);
  static const Color favorite = Color(0xFFF43F5E);

  static const LinearGradient primaryGradient = LinearGradient(
    colors: [primary, Color(0xFF7C3AED)],
    begin: Alignment.topLeft,
    end: Alignment.bottomRight,
  );

  static const LinearGradient weatherGradient = LinearGradient(
    colors: [Color(0xFF0EA5E9), Color(0xFF4F46E5)],
    begin: Alignment.topLeft,
    end: Alignment.bottomRight,
  );
}
```

lib/core/theme/app_theme.dart 107 lines

```dart
import 'package:flutter/material.dart';

import 'app_colors.dart';

class AppSpacing {
  AppSpacing._();

  static const double xs = 4;
  static const double sm = 8;
  static const double md = 16;
  static const double lg = 24;
  static const double xl = 32;

  static const double radius = 16;
```

```
}

class AppTheme {
  AppTheme._();

  static ThemeData get light {
    final colorScheme = ColorScheme.fromSeed(
      seedColor: AppColors.primary,
      primary: AppColors.primary,
      secondary: AppColors.accent,
      error: AppColors.error,
      surface: AppColors.surface,
    );

    final border = OutlineInputBorder(
      borderRadius: BorderRadius.circular(14),
      borderSide: const BorderSide(color: AppColors.border),
    );

    return ThemeData(
      useMaterial3: true,
      colorScheme: colorScheme,
      scaffoldBackgroundColor: AppColors.background,
      appBarTheme: const AppBarTheme(
        backgroundColor: AppColors.background,
        foregroundColor: AppColors.textPrimary,
        elevation: 0,
        scrolledUnderElevation: 0,
        centerTitle: false,
        titleTextStyle: TextStyle(
          color: AppColors.textPrimary,
          fontSize: 20,
          fontWeight: FontWeight.w700,
        ),
      ),
      textTheme: const TextTheme(
        headlineMedium: TextStyle(
          fontSize: 28,
          fontWeight: FontWeight.w800,
          color: AppColors.textPrimary,
        ),
        titleLarge: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.w700,
          color: AppColors.textPrimary,
        ),
        titleMedium: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w600,
          color: AppColors.textPrimary,
        ),
        bodyMedium: TextStyle(fontSize: 14, color: AppColors.textPrimary),
        bodySmall: TextStyle(fontSize: 13, color: AppColors.textSecondary),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: AppColors.surface,
        contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
        border: border,
        enabledBorder: border,
        focusedBorder: border.copyWith(
          borderSide: const BorderSide(color: AppColors.primary, width: 1.6),
        ),
        errorBorder: border.copyWith(
          borderSide: const BorderSide(color: AppColors.error),
        ),
        focusedErrorBorder: border.copyWith(
          borderSide: const BorderSide(color: AppColors.error, width: 1.6),
        ),
        hintStyle: const TextStyle(color: AppColors.textSecondary),
        prefixIconColor: AppColors.textSecondary,
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(
          minimumSize: const Size.fromHeight(52),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
          textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.w600),
        ),
      ),
      chipTheme: ChipThemeData(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
        side: const BorderSide(color: AppColors.border),
```

```
97          backgroundColor: AppColors.surface,
98          selectedColor: AppColors.primary,
99          // Without an explicit color, chips that don't set their own labelStyle
100         // (e.g. the recent-search ActionChips) render an invisible label.
101         labelStyle: const TextStyle(fontWeight: FontWeight.w500, color: AppColors.textPrimary),
102         showCheckmark: false,
103       ),
104       snackBarTheme: const SnackBarThemeData(behavior: SnackBarBehavior.floating),
105     );
106   }
107 }
```

lib/core/utils/formatters.dart 33 lines

```
1  class Formatters {
2    Formatters._();
3
4    /// 1428627663 -> "1.43B", 41798407 -> "41.8M", 12500 -> "12.5K".
5    static String compactNumber(int? value) {
6      if (value == null) return 'N/A';
7      if (value >= 1000000000) return '${_trim(value / 1000000000)}B';
8      if (value >= 1000000) return '${_trim(value / 1000000)}M';
9      if (value >= 1000) return '${_trim(value / 1000)}K';
10     return value.toString();
11   }
12
13   /// 41798407 -> "41,798,407".
14   static String groupedNumber(int? value) {
15     if (value == null) return 'N/A';
16     return value.toString().replaceAllMapped(
17           RegExp(r'\B(?=(\d{3})+(?!\d))'),
18           (_) => ',',
19         );
20   }
21
22   /// 20.5937 -> "20.5937° N".
23   static String latitude(double value) =>
24       '${value.abs().toStringAsFixed(4)}° ${value >= 0 ? 'N' : 'S'}';
25
26   static String longitude(double value) =>
27       '${value.abs().toStringAsFixed(4)}° ${value >= 0 ? 'E' : 'W'}';
28
29   static String _trim(double value) {
30     final fixed = value.toStringAsFixed(value >= 100 ? 0 : (value >= 10 ? 1 : 2));
31     return fixed.contains('.') ? fixed.replaceAll(RegExp(r'\.?0+$'), '') : fixed;
32   }
33 }
```

lib/core/utils/validators.dart 19 lines

```
1  class Validators {
2    Validators._();
3
4    static final RegExp _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+\.[\w.-]+$');
5
6    static String? email(String? value) {
7      final text = value?.trim() ?? '';
8      if (text.isEmpty) return 'Please enter your email';
9      if (!_emailRegex.hasMatch(text)) return 'Please enter a valid email';
10     return null;
11   }
12
13   static String? password(String? value) {
14     final text = value ?? '';
15     if (text.isEmpty) return 'Please enter your password';
16     if (text.length < 6) return 'Password must be at least 6 characters';
17     return null;
18   }
19 }
```

lib/core/utils/view_status.dart 2 lines

```
1  /// The common UI states every screen section can be in.
2  enum ViewStatus { loading, success, empty, error }
```

lib/data/models/country_model.dart 98 lines

```dart
/// A country as returned by REST Countries v5.
///
/// Every field is parsed defensively so missing or null values never crash the app.
class Country {
  final String code;
  final String name;
  final String officialName;
  final String capital;
  final String region;
  final String subregion;
  final String flagUrl;
  final String flagEmoji;
  final int? population;
  final List<Currency> currencies;
  final double? latitude;
  final double? longitude;

  const Country({
    required this.code,
    required this.name,
    required this.officialName,
    required this.capital,
    required this.region,
    required this.subregion,
    required this.flagUrl,
    required this.flagEmoji,
    required this.population,
    required this.currencies,
    required this.latitude,
    required this.longitude,
  });

  /// Latitude and longitude are both available, so weather and map can use them.
  bool get hasLocation => latitude != null && longitude != null;

  String get currencyLabel => currencies.isEmpty
      ? 'N/A'
      : currencies.map((c) => c.label).join(', ');

  factory Country.fromJson(Map<String, dynamic> json) {
    final names = _map(json['names']);
    final codes = _map(json['codes']);
    final flag = _map(json['flag']);
    final capitals = _list(json['capitals']).map(_map).toList();
    final firstCapital = capitals.isNotEmpty ? capitals.first : <String, dynamic>{};

    // Prefer the country's centre point; fall back to the capital's coordinates.
    final coordinates = _map(json['coordinates']).isNotEmpty
        ? _map(json['coordinates'])
        : _map(firstCapital['coordinates']);

    final name = _string(names['common'], fallback: 'Unknown');

    return Country(
      code: _string(codes['alpha_3'], fallback: name),
      name: name,
      officialName: _string(names['official'], fallback: name),
      capital: capitals.isEmpty
          ? 'N/A'
          : capitals.map((c) => _string(c['name'])).where((n) => n.isNotEmpty).join(', '),
      region: _string(json['region'], fallback: 'N/A'),
      subregion: _string(json['subregion']),
      flagUrl: _string(flag['url_png']),
      flagEmoji: _string(flag['emoji'], fallback: '\u{1F3F3}\u{FE0F}'),
      population: (json['population'] as num?)?.toInt(),
      currencies: _list(json['currencies']).map((c) => Currency.fromJson(_map(c))).toList(),
      latitude: (coordinates['lat'] as num?)?.toDouble(),
      longitude: (coordinates['lng'] as num?)?.toDouble(),
    );
  }

  static Map<String, dynamic> _map(dynamic value) =>
      value is Map ? Map<String, dynamic>.from(value) : <String, dynamic>{};

  static List<dynamic> _list(dynamic value) => value is List ? value : const [];

  static String _string(dynamic value, {String fallback = ''}) {
    final text = value?.toString().trim() ?? '';
    return text.isEmpty ? fallback : text;
```

```
  }
}

class Currency {
  final String code;
  final String name;
  final String symbol;

  const Currency({required this.code, required this.name, required this.symbol});

  /// e.g. "Indian rupee (₹)".
  String get label => symbol.isEmpty ? name : '$name ($symbol)';

  factory Currency.fromJson(Map<String, dynamic> json) => Currency(
        code: json['code']?.toString() ?? '',
        name: json['name']?.toString() ?? 'Unknown',
        symbol: json['symbol']?.toString() ?? '',
      );
}
```

lib/data/models/weather_model.dart 82 lines

```
import '../../core/network/app_exception.dart';

/// Current weather from the Open-Meteo `current` block.
class Weather {
  final double temperature;
  final double feelsLike;
  final int humidity;
  final double windSpeed;
  final int weatherCode;
  final bool isDay;

  const Weather({
    required this.temperature,
    required this.feelsLike,
    required this.humidity,
    required this.windSpeed,
    required this.weatherCode,
    required this.isDay,
  });

  factory Weather.fromJson(Map<String, dynamic> json) {
    final current = json['current'];
    if (current is! Map || current['temperature_2m'] == null) {
      throw const AppException.invalidResponse();
    }
    final temperature = (current['temperature_2m'] as num).toDouble();

    return Weather(
      temperature: temperature,
      feelsLike: (current['apparent_temperature'] as num?)?.toDouble() ?? temperature,
      humidity: (current['relative_humidity_2m'] as num?)?.toInt() ?? 0,
      windSpeed: (current['wind_speed_10m'] as num?)?.toDouble() ?? 0,
      weatherCode: (current['weather_code'] as num?)?.toInt() ?? -1,
      isDay: current['is_day'] != 0,
    );
  }

  /// Human readable condition for a WMO weather code.
  String get condition {
    switch (weatherCode) {
      case 0:
        return 'Clear sky';
      case 1:
        return 'Mainly clear';
      case 2:
        return 'Partly cloudy';
      case 3:
        return 'Overcast';
      case 45:
      case 48:
        return 'Foggy';
      case 51:
      case 53:
      case 55:
      case 56:
      case 57:
        return 'Drizzle';
      case 61:
      case 63:
```

```
      case 65:
      case 66:
      case 67:
      case 80:
      case 81:
      case 82:
        return 'Rain';
      case 71:
      case 73:
      case 75:
      case 77:
      case 85:
      case 86:
        return 'Snow';
      case 95:
      case 96:
      case 99:
        return 'Thunderstorm';
      default:
        return 'Unknown';
    }
  }
}
```

lib/data/repositories/country_repository.dart 52 lines

```
import '../../core/config/app_config.dart';
import '../../core/network/api_client.dart';
import '../../core/network/app_exception.dart';
import '../models/country_model.dart';

class CountryRepository {
  final ApiClient _client;

  CountryRepository(this._client);

  /// Fetches every country, following REST Countries' page-based pagination.
  Future<List<Country>> getCountries() async {
    if (AppConfig.countriesApiKey.isEmpty) {
      throw const AppException(
        'Countries API key is missing. Add COUNTRIES_API_KEY (see README).',
        type: AppErrorType.server,
      );
    }

    final countries = <Country>[];
    var offset = 0;
    var hasMore = true;
    const maxPages = 10; // Safety net against an endless loop.

    for (var page = 0; hasMore && page < maxPages; page++) {
      final json = await _client.get(
        AppConfig.countriesBaseUrl,
        query: {
          'response_fields': AppConfig.countriesFields,
          'limit': AppConfig.countriesPageSize,
          'offset': offset,
        },
        headers: {'Authorization': 'Bearer ${AppConfig.countriesApiKey}'},
      );

      final data = json is Map ? json['data'] : null;
      final objects = data is Map ? data['objects'] : null;
      if (objects is! List) throw const AppException.invalidResponse();

      countries.addAll(
        objects.whereType<Map>().map((e) => Country.fromJson(Map<String, dynamic>.from(e))),
      );

      final meta = data['meta'];
      hasMore = meta is Map && meta['more'] == true && objects.isNotEmpty;
      offset += objects.length;
    }

    countries.sort((a, b) => a.name.compareTo(b.name));
    return countries;
  }
}
```

lib/data/repositories/weather_repository.dart 24 lines

```dart
import '../../core/config/app_config.dart';
import '../../core/network/api_client.dart';
import '../../core/network/app_exception.dart';
import '../models/weather_model.dart';

class WeatherRepository {
  final ApiClient _client;

  WeatherRepository(this._client);

  Future<Weather> getCurrentWeather(double latitude, double longitude) async {
    final json = await _client.get(
      AppConfig.weatherBaseUrl,
      query: {
        'latitude': latitude,
        'longitude': longitude,
        'current': 'temperature_2m,apparent_temperature,relative_humidity_2m,'
            'wind_speed_10m,weather_code,is_day',
      },
    );
    if (json is! Map) throw const AppException.invalidResponse();
    return Weather.fromJson(Map<String, dynamic>.from(json));
  }
}
```

lib/data/services/auth_service.dart 58 lines

```dart
import 'package:firebase_auth/firebase_auth.dart';
import 'package:get/get.dart';

import '../../core/network/app_exception.dart';

/// Wraps Firebase Authentication and converts its errors into friendly messages.
class AuthService extends GetxService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  /// Firebase persists the session, so this is non-null after an app restart
  /// if the user was logged in.
  User? get currentUser => _auth.currentUser;

  bool get isLoggedIn => currentUser != null;

  Future<void> signIn(String email, String password) async {
    try {
      await _auth.signInWithEmailAndPassword(email: email, password: password);
    } on FirebaseAuthException catch (e) {
      throw AppException(_messageFor(e.code));
    }
  }

  Future<void> signUp(String email, String password) async {
    try {
      await _auth.createUserWithEmailAndPassword(email: email, password: password);
    } on FirebaseAuthException catch (e) {
      throw AppException(_messageFor(e.code));
    }
  }

  Future<void> signOut() => _auth.signOut();

  String _messageFor(String code) {
    switch (code) {
      case 'invalid-email':
        return 'The email address is not valid.';
      case 'user-disabled':
        return 'This account has been disabled.';
      case 'user-not-found':
      case 'wrong-password':
      case 'invalid-credential':
        return 'Incorrect email or password.';
      case 'email-already-in-use':
        return 'An account already exists for this email.';
      case 'weak-password':
        return 'Password is too weak. Use at least 6 characters.';
      case 'too-many-requests':
        return 'Too many attempts. Please try again later.';
      case 'network-request-failed':
        return 'No internet connection. Please check your network.';
```

```
      case 'operation-not-allowed':
        return 'Email/password sign-in is not enabled in Firebase.';
      default:
        return 'Authentication failed. Please try again.';
    }
  }
}
```

lib/data/services/storage_service.dart 61 lines

```
import 'package:get/get.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../../core/config/app_config.dart';

/// Persists favorites, recent searches and the last selected country
/// using SharedPreferences. Values are country codes (e.g. "IND").
///
/// The lists are reactive so every screen updates automatically.
class StorageService extends GetxService {
  static const _favoritesKey = 'favorite_countries';
  static const _recentKey = 'recent_searches';
  static const _lastSelectedKey = 'last_selected_country';

  late final SharedPreferences _prefs;

  final favorites = <String>[].obs;
  final recentSearches = <String>[].obs;
  final lastSelected = RxnString();

  Future<StorageService> init() async {
    _prefs = await SharedPreferences.getInstance();
    favorites.assignAll(_prefs.getStringList(_favoritesKey) ?? []);
    recentSearches.assignAll(_prefs.getStringList(_recentKey) ?? []);
    lastSelected.value = _prefs.getString(_lastSelectedKey);
    return this;
  }

  // Favorites

  bool isFavorite(String code) => favorites.contains(code);

  Future<void> toggleFavorite(String code) async {
    isFavorite(code) ? favorites.remove(code) : favorites.insert(0, code);
    await _prefs.setStringList(_favoritesKey, favorites);
  }

  // Recent searches (most recent first, max 5)

  Future<void> addRecentSearch(String code) async {
    recentSearches
      ..remove(code)
      ..insert(0, code);
    if (recentSearches.length > AppConfig.maxRecentSearches) {
      recentSearches.removeRange(AppConfig.maxRecentSearches, recentSearches.length);
    }
    await _prefs.setStringList(_recentKey, recentSearches);
  }

  Future<void> clearRecentSearches() async {
    recentSearches.clear();
    await _prefs.remove(_recentKey);
  }

  // Last selected country

  Future<void> saveLastSelected(String code) async {
    lastSelected.value = code;
    await _prefs.setString(_lastSelectedKey, code);
  }
}
```

lib/modules/login/login_controller.dart 61 lines

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../../core/network/app_exception.dart';
import '../../data/services/auth_service.dart';
import '../../routes/app_routes.dart';

class LoginController extends GetxController {
  final AuthService _auth = Get.find();

  final formKey = GlobalKey<FormState>();
  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  final isLoading = false.obs;
  final obscurePassword = true.obs;
  final isSignUpMode = false.obs;
  final errorMessage = ''.obs;

  void togglePasswordVisibility() => obscurePassword.toggle();

  void toggleMode() {
    isSignUpMode.toggle();
    errorMessage.value = '';
  }

  Future<void> submit() async {
    // The keyboard "done" action can call this while a request is running.
    if (isLoading.value) return;
    errorMessage.value = '';
    if (!(formKey.currentState?.validate() ?? false)) return;

    FocusManager.instance.primaryFocus?.unfocus();
    isLoading.value = true;

    final email = emailController.text.trim();
    final password = passwordController.text;

    try {
      if (isSignUpMode.value) {
        await _auth.signUp(email, password);
      } else {
        await _auth.signIn(email, password);
      }
      Get.offAllNamed(AppRoutes.home);
    } on AppException catch (e) {
      errorMessage.value = e.message;
      isLoading.value = false;
    } catch (_) {
      errorMessage.value = 'Something went wrong. Please try again.';
      isLoading.value = false;
    }
  }

  @override
  void onClose() {
    emailController.dispose();
    passwordController.dispose();
    super.onClose();
  }
}
```

lib/modules/login/login_view.dart 202 lines

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:get/get.dart';

import '../../core/config/app_config.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_theme.dart';
import '../../core/utils/validators.dart';
import 'login_controller.dart';

class LoginView extends GetView<LoginController> {
  const LoginView({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    // No AppBar here, so set light status bar icons ourselves for the dark gradient.
    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: SystemUiOverlayStyle.light,
      child: Scaffold(
        body: Container(
          decoration: const BoxDecoration(gradient: AppColors.primaryGradient),
          child: SafeArea(
            child: Center(
              child: SingleChildScrollView(
                padding: const EdgeInsets.all(AppSpacing.lg),
                child: ConstrainedBox(
                  constraints: const BoxConstraints(maxWidth: 440),
                  child: Column(
                    children: [
                      const _Header(),
                      const SizedBox(height: AppSpacing.xl),
                      _buildFormCard(context),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }

  Widget _buildFormCard(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return Container(
      padding: const EdgeInsets.all(AppSpacing.lg),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: BorderRadius.circular(24),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.12),
            blurRadius: 30,
            offset: const Offset(0, 12),
          ),
        ],
      ),
      child: Form(
        key: controller.formKey,
        // Re-check each field as the user types, so an old error like
        // "Please enter your email" disappears once the field is fixed.
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: Obx(
          () => Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                controller.isSignUpMode.value ? 'Create account' : 'Welcome back',
                style: textTheme.titleLarge,
              ),
              const SizedBox(height: AppSpacing.xs),
              Text(
                controller.isSignUpMode.value
                    ? 'Sign up to start exploring'
                    : 'Sign in to continue exploring',
                style: textTheme.bodySmall,
              ),
              const SizedBox(height: AppSpacing.lg),
              TextFormField(
                controller: controller.emailController,
                keyboardType: TextInputType.emailAddress,
                textInputAction: TextInputAction.next,
                autofillHints: const [AutofillHints.email],
                validator: Validators.email,
                decoration: const InputDecoration(
                  hintText: 'Email address',
                  prefixIcon: Icon(Icons.mail_outline_rounded),
                ),
              ),
              const SizedBox(height: AppSpacing.md),
              TextFormField(
                controller: controller.passwordController,
                obscureText: controller.obscurePassword.value,
                textInputAction: TextInputAction.done,
```

```
                autofillHints: const [AutofillHints.password],
                validator: Validators.password,
                onFieldSubmitted: (_) => controller.submit(),
                decoration: InputDecoration(
                  hintText: 'Password',
                  prefixIcon: const Icon(Icons.lock_outline_rounded),
                  suffixIcon: IconButton(
                    onPressed: controller.togglePasswordVisibility,
                    icon: Icon(
                      controller.obscurePassword.value
                          ? Icons.visibility_outlined
                          : Icons.visibility_off_outlined,
                    ),
                  ),
                ),
              ),
              if (controller.errorMessage.isNotEmpty) ...[
                const SizedBox(height: AppSpacing.md),
                _ErrorBanner(message: controller.errorMessage.value),
              ],
              const SizedBox(height: AppSpacing.lg),
              FilledButton(
                onPressed: controller.isLoading.value ? null : controller.submit,
                child: controller.isLoading.value
                    ? const SizedBox(
                        width: 22,
                        height: 22,
                        child: CircularProgressIndicator(strokeWidth: 2.5, color: Colors.white),
                      )
                    : Text(controller.isSignUpMode.value ? 'Create account' : 'Sign in'),
              ),
              const SizedBox(height: AppSpacing.sm),
              TextButton(
                onPressed: controller.isLoading.value ? null : controller.toggleMode,
                child: Text(
                  controller.isSignUpMode.value
                      ? 'Already have an account? Sign in'
                      : "Don't have an account? Create one",
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _Header extends StatelessWidget {
  const _Header();

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Container(
          padding: const EdgeInsets.all(AppSpacing.md),
          decoration: BoxDecoration(
            color: Colors.white.withValues(alpha: 0.18),
            borderRadius: BorderRadius.circular(24),
          ),
          child: const Icon(Icons.public_rounded, size: 56, color: Colors.white),
        ),
        const SizedBox(height: AppSpacing.md),
        const Text(
          AppConfig.appName,
          textAlign: TextAlign.center,
          style: TextStyle(color: Colors.white, fontSize: 26, fontWeight: FontWeight.w800),
        ),
        const SizedBox(height: AppSpacing.xs),
        Text(
          'Discover countries, live weather and maps',
          textAlign: TextAlign.center,
          style: TextStyle(color: Colors.white.withValues(alpha: 0.85), fontSize: 15),
        ),
      ],
    );
  }
}

class _ErrorBanner extends StatelessWidget {
  final String message;
```

```
179
180   const _ErrorBanner({required this.message});
181
182   @override
183   Widget build(BuildContext context) {
184     return Container(
185       padding: const EdgeInsets.all(12),
186       decoration: BoxDecoration(
187         color: AppColors.error.withValues(alpha: 0.08),
188         borderRadius: BorderRadius.circular(12),
189         border: Border.all(color: AppColors.error.withValues(alpha: 0.3)),
190       ),
191       child: Row(
192         children: [
193           const Icon(Icons.error_outline_rounded, color: AppColors.error, size: 20),
194           const SizedBox(width: AppSpacing.sm),
195           Expanded(
196             child: Text(message, style: const TextStyle(color: AppColors.error)),
197           ),
198         ],
199       ),
200     );
201   }
202 }
```

lib/modules/home/home_controller.dart 170 lines

```
1  import 'package:flutter/material.dart';
2  import 'package:get/get.dart';
3
4  import '../../core/network/app_exception.dart';
5  import '../../core/utils/view_status.dart';
6  import '../../data/models/country_model.dart';
7  import '../../data/repositories/country_repository.dart';
8  import '../../data/services/auth_service.dart';
9  import '../../data/services/storage_service.dart';
10 import '../../routes/app_routes.dart';
11
12 class HomeController extends GetxController {
13   final CountryRepository _repository = Get.find();
14   final AuthService _auth = Get.find();
15   final StorageService storage = Get.find();
16
17   static const String allRegions = 'All';
18
19   final searchController = TextEditingController();
20   final searchFocusNode = FocusNode();
21   final scrollController = ScrollController();
22
23   final status = ViewStatus.loading.obs;
24   final errorMessage = ''.obs;
25   final countries = <Country>[].obs; // Full list from the API.
26   final visibleCountries = <Country>[].obs; // After search & filters.
27   final searchQuery = ''.obs;
28   final selectedRegion = allRegions.obs;
29   final showFavoritesOnly = false.obs;
30
31   String get userEmail => _auth.currentUser?.email ?? '';
32
33   bool get isSearching => searchQuery.value.trim().isNotEmpty;
34
35   List<String> get regions {
36     final set = countries.map((c) => c.region).where((r) => r != 'N/A').toSet().toList()..sort();
37     return [allRegions, ...set];
38   }
39
40   List<Country> get recentCountries =>
41       storage.recentSearches.map(_findByCode).whereType<Country>().toList();
42
43   Country? get lastSelectedCountry {
44     final code = storage.lastSelected.value;
45     return code == null ? null : _findByCode(code);
46   }
47
48   @override
49   void onInit() {
50     super.onInit();
51     // Wait until the user stops typing before filtering.
52     debounce(
53       searchQuery,
54       (_) => _applyFiltersAndScrollToTop(),
```

```
      time: const Duration(milliseconds: 300),
    );
    ever(selectedRegion, (_) => _applyFiltersAndScrollToTop());
    ever(showFavoritesOnly, (_) => _applyFiltersAndScrollToTop());
    ever(storage.favorites, (_) => _applyFilters());
    fetchCountries();
  }

  /// [showLoader] is false for pull-to-refresh, which has its own spinner.
  Future<void> fetchCountries({bool showLoader = true}) async {
    if (showLoader) status.value = ViewStatus.loading;
    try {
      final result = await _repository.getCountries();
      countries.assignAll(result);
      status.value = result.isEmpty ? ViewStatus.empty : ViewStatus.success;
      _applyFilters();
    } on AppException catch (e) {
      _onFetchError(e.message);
    } catch (_) {
      _onFetchError('Unable to load countries. Please try again.');
    }
  }

  /// A failed pull-to-refresh keeps the list already on screen and shows a
  /// snackbar; only a failed first load replaces the list with the error view.
  void _onFetchError(String message) {
    if (countries.isNotEmpty) {
      status.value = ViewStatus.success;
      Get.closeAllSnackbars();
      Get.snackbar(
        'Could not refresh',
        message,
        snackPosition: SnackPosition.BOTTOM,
        margin: const EdgeInsets.all(16),
      );
      return;
    }
    errorMessage.value = message;
    status.value = ViewStatus.error;
  }

  Future<void> refreshCountries() => fetchCountries(showLoader: false);

  void onSearchChanged(String value) => searchQuery.value = value;

  void clearSearch() {
    searchController.clear();
    searchQuery.value = '';
    _applyFilters();
  }

  /// Closes the keyboard without clearing the search text.
  void hideKeyboard() => searchFocusNode.unfocus();

  void selectRegion(String region) {
    hideKeyboard();
    selectedRegion.value = region;
  }

  void toggleFavoritesOnly() {
    hideKeyboard();
    showFavoritesOnly.toggle();
  }

  void toggleFavorite(Country country) => storage.toggleFavorite(country.code);

  bool isFavorite(Country country) => storage.favorites.contains(country.code);

  void openCountry(Country country) {
    // A country opened from search results counts as a "recent search".
    if (isSearching) storage.addRecentSearch(country.code);
    // Unfocus first, otherwise the keyboard reopens when coming back.
    hideKeyboard();
    Get.toNamed(AppRoutes.details, arguments: country);
  }

  Future<void> logout() async {
    await _auth.signOut();
    Get.offAllNamed(AppRoutes.login);
  }

  void _applyFilters() {
```

```
    final query = searchQuery.value.trim().toLowerCase();

    final filtered = countries.where((country) {
      final matchesQuery =
          query.isEmpty ||
          country.name.toLowerCase().contains(query) ||
          country.officialName.toLowerCase().contains(query);
      final matchesRegion =
          selectedRegion.value == allRegions || country.region == selectedRegion.value;
      final matchesFavorite = !showFavoritesOnly.value || isFavorite(country);
      return matchesQuery && matchesRegion && matchesFavorite;
    });

    visibleCountries.assignAll(filtered);
  }

  /// New search/filter results should start from the top, not where the old
  /// list was scrolled. Not used for favorite toggles, so tapping a heart
  /// mid-list doesn't jump.
  void _applyFiltersAndScrollToTop() {
    _applyFilters();
    if (scrollController.hasClients) scrollController.jumpTo(0);
  }

  Country? _findByCode(String code) => countries.firstWhereOrNull((c) => c.code == code);

  @override
  void onClose() {
    searchController.dispose();
    searchFocusNode.dispose();
    scrollController.dispose();
    super.onClose();
  }
}
```

lib/modules/home/home_view.dart 272 lines

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../../core/theme/app_colors.dart';
import '../../core/theme/app_theme.dart';
import '../../core/utils/view_status.dart';
import '../../data/models/country_model.dart';
import '../../widgets/status_view.dart';
import 'home_controller.dart';
import 'widgets/country_card.dart';
import 'widgets/country_list_skeleton.dart';
import 'widgets/last_viewed_card.dart';
import 'widgets/recent_searches.dart';

class HomeView extends GetView<HomeController> {
  const HomeView({super.key});

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: controller.hideKeyboard,
      child: Scaffold(
        appBar: AppBar(
          toolbarHeight: 72,
          title: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text('Explore countries'),
              Text(controller.userEmail, style: Theme.of(context).textTheme.bodySmall),
            ],
          ),
          actions: [
            IconButton(
              tooltip: 'Logout',
              onPressed: _confirmLogout,
              icon: const Icon(Icons.logout_rounded),
            ),
            const SizedBox(width: AppSpacing.sm),
          ],
        ),
        body: Column(
          children: [
            _buildSearchField(),
            _buildFilters(),
```

```
            const SizedBox(height: AppSpacing.sm),
            Expanded(child: Obx(_buildBody)),
          ],
        ),
      ),
    );
  }

  Widget _buildSearchField() {
    return Padding(
      padding: const EdgeInsets.fromLTRB(AppSpacing.md, 0, AppSpacing.md, 12),
      child: Obx(
        () => TextField(
          controller: controller.searchController,
          focusNode: controller.searchFocusNode,
          onChanged: controller.onSearchChanged,
          onSubmitted: (_) => controller.hideKeyboard(),
          onTapOutside: (_) => controller.hideKeyboard(),
          textInputAction: TextInputAction.search,
          decoration: InputDecoration(
            hintText: 'Search countries by name',
            prefixIcon: const Icon(Icons.search_rounded),
            suffixIcon: controller.searchQuery.isEmpty
                ? null
                : IconButton(
                    tooltip: 'Clear search',
                    onPressed: controller.clearSearch,
                    icon: const Icon(Icons.close_rounded),
                  ),
          ),
        ),
      ),
    );
  }

  Widget _buildFilters() {
    return SizedBox(
      height: 40,
      child: Obx(() {
        final regions = controller.regions;
        return ListView(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md),
          children: [
            FilterChip(
              avatar: Icon(
                Icons.favorite_rounded,
                size: 18,
                color: controller.showFavoritesOnly.value ? Colors.white : AppColors.favorite,
              ),
              label: Text('Favorites (${controller.storage.favorites.length})'),
              selected: controller.showFavoritesOnly.value,
              selectedColor: AppColors.favorite,
              labelStyle: TextStyle(
                color: controller.showFavoritesOnly.value ? Colors.white : AppColors.textPrimary,
              ),
              onSelected: (_) => controller.toggleFavoritesOnly(),
            ),
            const SizedBox(width: AppSpacing.sm),
            for (final region in regions) ...[
              ChoiceChip(
                label: Text(region),
                selected: controller.selectedRegion.value == region,
                labelStyle: TextStyle(
                  color: controller.selectedRegion.value == region
                      ? Colors.white
                      : AppColors.textPrimary,
                ),
                onSelected: (_) => controller.selectRegion(region),
              ),
              const SizedBox(width: AppSpacing.sm),
            ],
          ],
        );
      }),
    );
  }

  Widget _buildBody() {
    switch (controller.status.value) {
      case ViewStatus.loading:
        return const CountryListSkeleton();
```

```
      case ViewStatus.error:
        return StatusView.error(
          title: 'Could not load countries',
          message: controller.errorMessage.value,
          onAction: controller.fetchCountries,
        );
      case ViewStatus.empty:
        return StatusView(
          icon: Icons.public_off_rounded,
          title: 'No countries found',
          message: 'The server returned no countries. Please try again.',
          actionLabel: 'Reload',
          onAction: controller.fetchCountries,
        );
      case ViewStatus.success:
        return _buildCountryList();
    }
  }

  Widget _buildCountryList() {
    final countries = controller.visibleCountries;
    final showExtras = !controller.isSearching && !controller.showFavoritesOnly.value;
    final lastViewed = showExtras ? controller.lastSelectedCountry : null;
    final recents = showExtras ? controller.recentCountries : const <Country>[];
    final favorites = controller.storage.favorites.toSet();

    return RefreshIndicator(
      onRefresh: controller.refreshCountries,
      child: CustomScrollView(
        controller: controller.scrollController,
        physics: const AlwaysScrollableScrollPhysics(),
        keyboardDismissBehavior: ScrollViewKeyboardDismissBehavior.onDrag,
        slivers: [
          if (lastViewed != null)
            SliverToBoxAdapter(
              child: Padding(
                padding: const EdgeInsets.only(bottom: AppSpacing.sm),
                child: LastViewedCard(
                  country: lastViewed,
                  onTap: () => controller.openCountry(lastViewed),
                ),
              ),
            ),
          if (recents.isNotEmpty)
            SliverToBoxAdapter(
              child: Padding(
                padding: const EdgeInsets.only(bottom: AppSpacing.sm),
                child: RecentSearches(
                  countries: recents,
                  onTap: controller.openCountry,
                  onClear: controller.storage.clearRecentSearches,
                ),
              ),
            ),
          SliverToBoxAdapter(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(AppSpacing.md, AppSpacing.sm, AppSpacing.md, 12),
              child: Text(
                '${countries.length} ${countries.length == 1 ? 'country' : 'countries'}',
                style: const TextStyle(color: AppColors.textSecondary, fontWeight: FontWeight.w600),
              ),
            ),
          ),
          if (countries.isEmpty)
            SliverFillRemaining(hasScrollBody: false, child: _buildNoResults())
          else
            // Keeps the last card above the Android navigation bar (edge-to-edge).
            SliverSafeArea(
              top: false,
              sliver: SliverPadding(
                padding: const EdgeInsets.fromLTRB(AppSpacing.md, 0, AppSpacing.md, AppSpacing.lg),
                // Max extent makes the grid show 1 column on phones, 2+ on tablets.
                sliver: SliverGrid.builder(
                  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
                    maxCrossAxisExtent: 520,
                    mainAxisExtent: 84,
                    mainAxisSpacing: 12,
                    crossAxisSpacing: 12,
                  ),
                  itemCount: countries.length,
                  itemBuilder: (_, index) {
                    final country = countries[index];
```

```
                    return CountryCard(
                      country: country,
                      isFavorite: favorites.contains(country.code),
                      onTap: () => controller.openCountry(country),
                      onFavoriteTap: () => controller.toggleFavorite(country),
                    );
                  },
                ),
              ),
            ),
        ],
      ),
    );
  }

  Widget _buildNoResults() {
    if (controller.isSearching) {
      return StatusView(
        icon: Icons.search_off_rounded,
        title: 'No results',
        message:
            'No country matches "${controller.searchQuery.value.trim()}". '
            'Try a different name.',
        actionLabel: 'Clear search',
        onAction: controller.clearSearch,
      );
    }
    if (controller.showFavoritesOnly.value) {
      return const StatusView(
        icon: Icons.favorite_border_rounded,
        title: 'No favorites yet',
        message: 'Tap the heart on any country to save it here.',
        color: AppColors.favorite,
      );
    }
    return StatusView(
      icon: Icons.filter_alt_off_rounded,
      title: 'Nothing here',
      message: 'No countries found for this region.',
      actionLabel: 'Show all',
      onAction: () => controller.selectRegion(HomeController.allRegions),
    );
  }

  void _confirmLogout() {
    Get.dialog(
      AlertDialog(
        title: const Text('Log out?'),
        content: const Text('You will need to sign in again to explore countries.'),
        actions: [
          TextButton(onPressed: () => Get.back(), child: const Text('Cancel')),
          FilledButton(
            style: FilledButton.styleFrom(minimumSize: const Size(100, 44)),
            onPressed: () {
              Get.back();
              controller.logout();
            },
            child: const Text('Log out'),
          ),
        ],
      ),
    );
  }
}
```

lib/modules/home/widgets/country_card.dart 115 lines

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_theme.dart';
import '../../../data/models/country_model.dart';
import '../../../widgets/flag_image.dart';

class CountryCard extends StatelessWidget {
  final Country country;
  final bool isFavorite;
  final VoidCallback onTap;
  final VoidCallback onFavoriteTap;

  const CountryCard({
```

```
    super.key,
    required this.country,
    required this.isFavorite,
    required this.onTap,
    required this.onFavoriteTap,
  });

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return Material(
      color: AppColors.surface,
      borderRadius: BorderRadius.circular(AppSpacing.radius),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(AppSpacing.radius),
        child: Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(AppSpacing.radius),
            border: Border.all(color: AppColors.border),
          ),
          child: Row(
            children: [
              Hero(
                tag: 'flag-${country.code}',
                child: FlagImage(url: country.flagUrl, emoji: country.flagEmoji),
              ),
              const SizedBox(width: AppSpacing.md),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Text(
                      country.name,
                      style: textTheme.titleMedium,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                    ),
                    const SizedBox(height: 4),
                    Row(
                      children: [
                        const Icon(
                          Icons.location_city_rounded,
                          size: 14,
                          color: AppColors.textSecondary,
                        ),
                        const SizedBox(width: 4),
                        Flexible(
                          child: Text(
                            country.capital,
                            style: textTheme.bodySmall,
                            maxLines: 1,
                            overflow: TextOverflow.ellipsis,
                          ),
                        ),
                        const SizedBox(width: AppSpacing.sm),
                        RegionBadge(region: country.region),
                      ],
                    ),
                  ],
                ),
              ),
              IconButton(
                onPressed: onFavoriteTap,
                tooltip: isFavorite ? 'Remove from favorites' : 'Add to favorites',
                icon: Icon(
                  isFavorite ? Icons.favorite_rounded : Icons.favorite_border_rounded,
                  color: isFavorite ? AppColors.favorite : AppColors.textSecondary,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class RegionBadge extends StatelessWidget {
```

```
  final String region;

  const RegionBadge({super.key, required this.region});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
      decoration: BoxDecoration(
        color: AppColors.primary.withValues(alpha: 0.08),
        borderRadius: BorderRadius.circular(20),
      ),
      child: Text(
        region,
        style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w600, color: AppColors.primary),
      ),
    );
  }
}
```

lib/modules/home/widgets/country_list_skeleton.dart 51 lines

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_theme.dart';

/// Placeholder cards shown while countries are loading.
class CountryListSkeleton extends StatelessWidget {
  const CountryListSkeleton({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.separated(
      padding: const EdgeInsets.all(AppSpacing.md),
      physics: const NeverScrollableScrollPhysics(),
      itemCount: 8,
      separatorBuilder: (_, _) => const SizedBox(height: 12),
      itemBuilder: (_, _) => Container(
        height: 70,
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: AppColors.surface,
          borderRadius: BorderRadius.circular(AppSpacing.radius),
          border: Border.all(color: AppColors.border),
        ),
        child: Row(
          children: [
            _block(width: 64, height: 44),
            const SizedBox(width: AppSpacing.md),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  _block(width: 140, height: 14),
                  const SizedBox(height: 8),
                  _block(width: 90, height: 10),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }

  Widget _block({required double width, required double height}) => Container(
    width: width,
    height: height,
    decoration: BoxDecoration(color: AppColors.border, borderRadius: BorderRadius.circular(8)),
  );
}
```

lib/modules/home/widgets/last_viewed_card.dart 66 lines

```dart
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_theme.dart';
import '../../../data/models/country_model.dart';
import '../../../widgets/flag_image.dart';

/// Gradient banner that lets the user jump back to the last viewed country.
class LastViewedCard extends StatelessWidget {
  final Country country;
  final VoidCallback onTap;

  const LastViewedCard({super.key, required this.country, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md),
      child: Material(
        borderRadius: BorderRadius.circular(AppSpacing.radius),
        clipBehavior: Clip.antiAlias,
        child: Ink(
          decoration: const BoxDecoration(gradient: AppColors.primaryGradient),
          child: InkWell(
            onTap: onTap,
            child: Padding(
              padding: const EdgeInsets.all(AppSpacing.md),
              child: Row(
                children: [
                  FlagImage(url: country.flagUrl, emoji: country.flagEmoji, width: 56, height: 40),
                  const SizedBox(width: AppSpacing.md),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          'Continue exploring',
                          style: TextStyle(
                            color: Colors.white.withValues(alpha: 0.8),
                            fontSize: 12,
                          ),
                        ),
                        const SizedBox(height: 2),
                        Text(
                          country.name,
                          maxLines: 1,
                          overflow: TextOverflow.ellipsis,
                          style: const TextStyle(
                            color: Colors.white,
                            fontSize: 17,
                            fontWeight: FontWeight.w700,
                          ),
                        ),
                      ],
                    ),
                  ),
                  const Icon(Icons.arrow_forward_rounded, color: Colors.white),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/modules/home/widgets/recent_searches.dart 58 lines

```dart
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_theme.dart';
import '../../../data/models/country_model.dart';

/// Horizontal chips for the last 5 countries opened from search.
class RecentSearches extends StatelessWidget {
  final List<Country> countries;
```

```
  final ValueChanged<Country> onTap;
  final VoidCallback onClear;

  const RecentSearches({
    super.key,
    required this.countries,
    required this.onTap,
    required this.onClear,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md),
          child: Row(
            children: [
              const Icon(Icons.history_rounded, size: 18, color: AppColors.textSecondary),
              const SizedBox(width: 6),
              Expanded(
                child: Text('Recent searches', style: Theme.of(context).textTheme.titleMedium),
              ),
              TextButton(onPressed: onClear, child: const Text('Clear')),
            ],
          ),
        ),
        SizedBox(
          height: 40,
          child: ListView.separated(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md),
            itemCount: countries.length,
            separatorBuilder: (_, _) => const SizedBox(width: AppSpacing.sm),
            itemBuilder: (_, index) {
              final country = countries[index];
              return ActionChip(
                avatar: Text(country.flagEmoji),
                label: Text(country.name, style: const TextStyle(color: AppColors.textPrimary)),
                onPressed: () => onTap(country),
              );
            },
          ),
        ),
      ],
    );
  }
}
```

lib/modules/details/country_details_controller.dart 59 lines

```
import 'package:get/get.dart';

import '../../core/network/app_exception.dart';
import '../../core/utils/view_status.dart';
import '../../data/models/country_model.dart';
import '../../data/models/weather_model.dart';
import '../../data/repositories/weather_repository.dart';
import '../../data/services/storage_service.dart';

class CountryDetailsController extends GetxController {
  final WeatherRepository _weatherRepository = Get.find();
  final StorageService _storage = Get.find();

  /// The selected country passed via `Get.toNamed(..., arguments: country)`.
  final Country? country = Get.arguments is Country ? Get.arguments as Country : null;

  // Weather has its own state so the rest of the screen stays usable if it fails.
  final weatherStatus = ViewStatus.loading.obs;
  final weather = Rxn<Weather>();
  final weatherError = ''.obs;

  bool get isFavorite => country != null && _storage.favorites.contains(country!.code);

  @override
  void onInit() {
    super.onInit();
    if (country == null) return;
    _storage.saveLastSelected(country!.code);
    fetchWeather();
```

```
  }

  Future<void> fetchWeather() async {
    final selected = country;
    // No coordinates means we can't ask for weather - show the empty state instead.
    if (selected == null || !selected.hasLocation) {
      weatherStatus.value = ViewStatus.empty;
      return;
    }

    weatherStatus.value = ViewStatus.loading;
    try {
      weather.value = await _weatherRepository.getCurrentWeather(
        selected.latitude!,
        selected.longitude!,
      );
      weatherStatus.value = ViewStatus.success;
    } on AppException catch (e) {
      weatherError.value = e.message;
      weatherStatus.value = ViewStatus.error;
    } catch (_) {
      weatherError.value = 'Unable to load weather right now.';
      weatherStatus.value = ViewStatus.error;
    }
  }

  void toggleFavorite() {
    if (country != null) _storage.toggleFavorite(country!.code);
  }
}
```

lib/modules/details/country_details_view.dart 215 lines

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../../core/theme/app_colors.dart';
import '../../core/theme/app_theme.dart';
import '../../core/utils/formatters.dart';
import '../../data/models/country_model.dart';
import '../../widgets/flag_image.dart';
import '../../widgets/section_card.dart';
import '../../widgets/status_view.dart';
import 'country_details_controller.dart';
import 'widgets/info_tile.dart';
import 'widgets/map_section.dart';
import 'widgets/weather_section.dart';

class CountryDetailsView extends GetView<CountryDetailsController> {
  const CountryDetailsView({super.key});

  @override
  Widget build(BuildContext context) {
    final country = controller.country;
    if (country == null) {
      return Scaffold(
        appBar: AppBar(),
        body: StatusView(
          icon: Icons.travel_explore_rounded,
          title: 'Country not found',
          message: 'Please go back and select a country again.',
          actionLabel: 'Go back',
          onAction: () => Get.back(),
        ),
      );
    }

    return Scaffold(
      body: CustomScrollView(
        slivers: [
          _buildAppBar(country),
          // Keeps the map card above the Android navigation bar (edge-to-edge).
          SliverSafeArea(
            top: false,
            sliver: SliverToBoxAdapter(
              child: Center(
                child: ConstrainedBox(
                  constraints: const BoxConstraints(maxWidth: 1100),
                  child: Padding(
                    padding: const EdgeInsets.all(AppSpacing.md),
                    child: LayoutBuilder(
```

```
                    builder: (context, constraints) {
                      final isWide = constraints.maxWidth >= 800;
                      final facts = _buildFacts(context, country);
                      final weatherAndMap = Column(
                        children: [
                          _buildWeather(),
                          const SizedBox(height: AppSpacing.md),
                          SectionCard(
                            title: 'Location',
                            icon: Icons.map_rounded,
                            child: MapSection(country: country),
                          ),
                        ],
                      );

                      // Tablets / wide screens: two columns. Phones: one column.
                      if (isWide) {
                        return Row(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Expanded(child: facts),
                            const SizedBox(width: AppSpacing.md),
                            Expanded(child: weatherAndMap),
                          ],
                        );
                      }
                      return Column(
                        children: [
                          facts,
                          const SizedBox(height: AppSpacing.md),
                          weatherAndMap,
                        ],
                      );
                    },
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildAppBar(Country country) {
    return SliverAppBar(
      pinned: true,
      expandedHeight: 230,
      backgroundColor: AppColors.primaryDark,
      foregroundColor: Colors.white,
      actions: [
        Obx(
          () => IconButton(
            tooltip: controller.isFavorite ? 'Remove from favorites' : 'Add to favorites',
            onPressed: controller.toggleFavorite,
            icon: Icon(
              controller.isFavorite ? Icons.favorite_rounded : Icons.favorite_border_rounded,
              color: controller.isFavorite ? AppColors.favorite : Colors.white,
            ),
          ),
        ),
        const SizedBox(width: AppSpacing.sm),
      ],
      flexibleSpace: FlexibleSpaceBar(
        title: Text(
          country.name,
          style: const TextStyle(color: Colors.white, fontWeight: FontWeight.w700),
        ),
        background: Stack(
          fit: StackFit.expand,
          children: [
            Hero(
              tag: 'flag-${country.code}',
              child: FlagImage(
                url: country.flagUrl,
                emoji: country.flagEmoji,
                width: double.infinity,
                height: double.infinity,
                radius: 0,
              ),
            ),
```

```
            // Dark gradient so the title stays readable on light flags.
            const DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  begin: Alignment.topCenter,
                  end: Alignment.bottomCenter,
                  colors: [Colors.black38, Colors.transparent, Colors.black87],
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }

  Widget _buildFacts(BuildContext context, Country country) {
    final tiles = [
      InfoTile(icon: Icons.location_city_rounded, label: 'Capital', value: country.capital),
      InfoTile(icon: Icons.public_rounded, label: 'Region', value: country.region),
      InfoTile(
        icon: Icons.groups_rounded,
        label: 'Population',
        value: Formatters.groupedNumber(country.population),
        singleLine: true,
      ),
      InfoTile(icon: Icons.payments_rounded, label: 'Currency', value: country.currencyLabel),
      InfoTile(
        icon: Icons.north_rounded,
        label: 'Latitude',
        value: country.latitude == null ? 'N/A' : Formatters.latitude(country.latitude!),
        singleLine: true,
      ),
      InfoTile(
        icon: Icons.east_rounded,
        label: 'Longitude',
        value: country.longitude == null ? 'N/A' : Formatters.longitude(country.longitude!),
        singleLine: true,
      ),
    ];

    return SectionCard(
      title: 'Country facts',
      icon: Icons.info_outline_rounded,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(country.officialName, style: Theme.of(context).textTheme.titleMedium),
          if (country.subregion.isNotEmpty) ...[
            const SizedBox(height: 2),
            Text(country.subregion, style: Theme.of(context).textTheme.bodySmall),
          ],
          const SizedBox(height: AppSpacing.md),
          LayoutBuilder(
            builder: (context, constraints) {
              // Two tiles per row, or one on very narrow screens.
              final columns = constraints.maxWidth < 320 ? 1 : 2;
              final tileWidth = (constraints.maxWidth - (columns - 1) * AppSpacing.sm) / columns;
              return Wrap(
                spacing: AppSpacing.sm,
                runSpacing: AppSpacing.sm,
                children: [for (final tile in tiles) SizedBox(width: tileWidth, child: tile)],
              );
            },
          ),
        ],
      ),
    );
  }

  Widget _buildWeather() {
    return SectionCard(
      title: 'Weather',
      icon: Icons.wb_sunny_outlined,
      child: Obx(
        () => WeatherSection(
          status: controller.weatherStatus.value,
          weather: controller.weather.value,
          errorMessage: controller.weatherError.value,
          onRetry: controller.fetchWeather,
        ),
      ),
```

```
    );
  }
}
```

lib/modules/details/widgets/info_tile.dart 65 lines

```dart
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';

/// Small icon + label + value block used in the country facts grid.
class InfoTile extends StatelessWidget {
  static const _valueStyle = TextStyle(fontWeight: FontWeight.w600, fontSize: 14);

  final IconData icon;
  final String label;
  final String value;

  /// Keeps values like "1,429,404,000" on one line by scaling them down
  /// instead of breaking them in the middle of the number.
  final bool singleLine;

  const InfoTile({
    super.key,
    required this.icon,
    required this.label,
    required this.value,
    this.singleLine = false,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: AppColors.background,
        borderRadius: BorderRadius.circular(12),
      ),
      child: Row(
        children: [
          Container(
            padding: const EdgeInsets.all(8),
            decoration: BoxDecoration(
              color: AppColors.primary.withValues(alpha: 0.1),
              borderRadius: BorderRadius.circular(10),
            ),
            child: Icon(icon, size: 20, color: AppColors.primary),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(label, style: Theme.of(context).textTheme.bodySmall),
                const SizedBox(height: 2),
                if (singleLine)
                  FittedBox(
                    fit: BoxFit.scaleDown,
                    alignment: Alignment.centerLeft,
                    child: Text(value, maxLines: 1, style: _valueStyle),
                  )
                else
                  Text(value, maxLines: 2, overflow: TextOverflow.ellipsis, style: _valueStyle),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

lib/modules/details/widgets/map_section.dart 80 lines

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_theme.dart';
import '../../../data/models/country_model.dart';
import '../../../widgets/country_map.dart';

/// Small map preview; tapping "Expand" opens a full-screen interactive map.
class MapSection extends StatelessWidget {
  final Country country;

  const MapSection({super.key, required this.country});

  @override
  Widget build(BuildContext context) {
    if (!country.hasLocation) {
      return Container(
        height: 160,
        alignment: Alignment.center,
        decoration: BoxDecoration(
          color: AppColors.background,
          borderRadius: BorderRadius.circular(12),
        ),
        child: const Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(Icons.map_outlined, size: 36, color: AppColors.textSecondary),
            SizedBox(height: AppSpacing.sm),
            Text('Location not available', style: TextStyle(color: AppColors.textSecondary)),
          ],
        ),
      );
    }

    return ClipRRect(
      borderRadius: BorderRadius.circular(12),
      child: SizedBox(
        height: 240,
        child: Stack(
          children: [
            CountryMap(
              latitude: country.latitude!,
              longitude: country.longitude!,
              interactive: false,
            ),
            Positioned(
              right: 8,
              top: 8,
              child: FilledButton.tonalIcon(
                onPressed: () => Get.to(() => FullScreenMap(country: country)),
                style: FilledButton.styleFrom(minimumSize: const Size(0, 36)),
                icon: const Icon(Icons.open_in_full_rounded, size: 18),
                label: const Text('Expand'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class FullScreenMap extends StatelessWidget {
  final Country country;

  const FullScreenMap({super.key, required this.country});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(country.name)),
      body: CountryMap(
        latitude: country.latitude!,
        longitude: country.longitude!,
        zoom: 5,
      ),
    );
  }
```

```
80  }
```

lib/modules/details/widgets/weather_section.dart 230 lines

```
 1  import 'package:flutter/material.dart';
 2
 3  import '../../../core/theme/app_colors.dart';
 4  import '../../../core/theme/app_theme.dart';
 5  import '../../../core/utils/view_status.dart';
 6  import '../../../data/models/weather_model.dart';
 7
 8  /// Current weather card with its own loading / error / empty states.
 9  class WeatherSection extends StatelessWidget {
10    final ViewStatus status;
11    final Weather? weather;
12    final String errorMessage;
13    final VoidCallback onRetry;
14
15    const WeatherSection({
16      super.key,
17      required this.status,
18      required this.weather,
19      required this.errorMessage,
20      required this.onRetry,
21    });
22
23    @override
24    Widget build(BuildContext context) {
25      return AnimatedSwitcher(
26        duration: const Duration(milliseconds: 250),
27        child: switch (status) {
28          ViewStatus.loading => const _MessageCard(
29              key: ValueKey('loading'),
30              icon: Icons.cloud_sync_rounded,
31              message: 'Fetching current weather...',
32              showLoader: true,
33            ),
34          ViewStatus.error => _MessageCard(
35              key: const ValueKey('error'),
36              icon: Icons.cloud_off_rounded,
37              message: errorMessage,
38              color: AppColors.error,
39              onRetry: onRetry,
40            ),
41          ViewStatus.empty => const _MessageCard(
42              key: ValueKey('empty'),
43              icon: Icons.location_off_rounded,
44              message: 'Weather is not available because this country has no coordinates.',
45            ),
46          ViewStatus.success => weather == null
47              ? const SizedBox.shrink()
48              : _WeatherCard(key: const ValueKey('success'), weather: weather!),
49        },
50      );
51    }
52  }
53
54  class _WeatherCard extends StatelessWidget {
55    final Weather weather;
56
57    const _WeatherCard({super.key, required this.weather});
58
59    @override
60    Widget build(BuildContext context) {
61      return Container(
62        width: double.infinity,
63        padding: const EdgeInsets.all(20),
64        decoration: BoxDecoration(
65          gradient: AppColors.weatherGradient,
66          borderRadius: BorderRadius.circular(AppSpacing.radius),
67        ),
68        child: Column(
69          crossAxisAlignment: CrossAxisAlignment.start,
70          children: [
71            Row(
72              children: [
73                Expanded(
74                  child: Column(
75                    crossAxisAlignment: CrossAxisAlignment.start,
76                    children: [
77                      const Text(
```

```
                      'Current weather',
                      style: TextStyle(color: Colors.white70, fontSize: 13),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      '${weather.temperature.round()}°C',
                      style: const TextStyle(
                        color: Colors.white,
                        fontSize: 44,
                        fontWeight: FontWeight.w800,
                      ),
                    ),
                    Text(
                      '${weather.condition} · Feels like ${weather.feelsLike.round()}°C',
                      style: const TextStyle(color: Colors.white, fontSize: 15),
                    ),
                  ],
                ),
              ),
              Icon(_iconFor(weather), size: 72, color: Colors.white),
            ],
          ),
          const SizedBox(height: 20),
          Row(
            children: [
              Expanded(
                child: _WeatherStat(
                  icon: Icons.water_drop_rounded,
                  label: 'Humidity',
                  value: '${weather.humidity}%',
                ),
              ),
              const SizedBox(width: 12),
              Expanded(
                child: _WeatherStat(
                  icon: Icons.air_rounded,
                  label: 'Wind speed',
                  value: '${weather.windSpeed.toStringAsFixed(1)} km/h',
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }

  IconData _iconFor(Weather weather) {
    final code = weather.weatherCode;
    if (code == 0 || code == 1) {
      return weather.isDay ? Icons.wb_sunny_rounded : Icons.nightlight_round;
    }
    if (code == 2) return Icons.wb_cloudy_rounded;
    if (code == 3) return Icons.cloud_rounded;
    if (code == 45 || code == 48) return Icons.foggy;
    if (code >= 51 && code <= 67) return Icons.grain_rounded;
    if (code >= 71 && code <= 77) return Icons.ac_unit_rounded;
    if (code >= 80 && code <= 82) return Icons.umbrella_rounded;
    if (code == 85 || code == 86) return Icons.ac_unit_rounded;
    if (code >= 95) return Icons.thunderstorm_rounded;
    return Icons.cloud_queue_rounded;
  }
}

class _WeatherStat extends StatelessWidget {
  final IconData icon;
  final String label;
  final String value;

  const _WeatherStat({required this.icon, required this.label, required this.value});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.white.withValues(alpha: 0.18),
        borderRadius: BorderRadius.circular(12),
      ),
      child: Row(
        children: [
          Icon(icon, color: Colors.white, size: 22),
```

```
            const SizedBox(width: AppSpacing.sm),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
                  Text(
                    value,
                    style: const TextStyle(
                      color: Colors.white,
                      fontSize: 15,
                      fontWeight: FontWeight.w700,
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      );
    }
  }

  class _MessageCard extends StatelessWidget {
    final IconData icon;
    final String message;
    final Color color;
    final bool showLoader;
    final VoidCallback? onRetry;

    const _MessageCard({
      super.key,
      required this.icon,
      required this.message,
      this.color = AppColors.accent,
      this.showLoader = false,
      this.onRetry,
    });

    @override
    Widget build(BuildContext context) {
      return Container(
        width: double.infinity,
        padding: const EdgeInsets.all(AppSpacing.md),
        decoration: BoxDecoration(
          color: color.withValues(alpha: 0.08),
          borderRadius: BorderRadius.circular(AppSpacing.radius),
          border: Border.all(color: color.withValues(alpha: 0.25)),
        ),
        child: Row(
          children: [
            showLoader
                ? SizedBox(
                    width: 24,
                    height: 24,
                    child: CircularProgressIndicator(strokeWidth: 2.5, color: color),
                  )
                : Icon(icon, color: color),
            const SizedBox(width: 12),
            Expanded(child: Text(message)),
            if (onRetry != null)
              TextButton.icon(
                onPressed: onRetry,
                icon: const Icon(Icons.refresh_rounded, size: 18),
                label: const Text('Retry'),
              ),
          ],
        ),
      );
    }
  }
```

lib/routes/app_routes.dart 7 lines

```
class AppRoutes {
  AppRoutes._();

  static const String login = '/login';
  static const String home = '/home';
  static const String details = '/details';
}
```

lib/routes/app_pages.dart 33 lines

```dart
import 'package:get/get.dart';

import '../modules/details/country_details_controller.dart';
import '../modules/details/country_details_view.dart';
import '../modules/home/home_controller.dart';
import '../modules/home/home_view.dart';
import '../modules/login/login_controller.dart';
import '../modules/login/login_view.dart';
import 'app_routes.dart';

/// Each page registers its own controller, so controllers are created when
/// the page opens and disposed when it closes.
class AppPages {
  AppPages._();

  static final pages = [
    GetPage(
      name: AppRoutes.login,
      page: () => const LoginView(),
      binding: BindingsBuilder(() => Get.lazyPut(() => LoginController())),
    ),
    GetPage(
      name: AppRoutes.home,
      page: () => const HomeView(),
      binding: BindingsBuilder(() => Get.lazyPut(() => HomeController())),
    ),
    GetPage(
      name: AppRoutes.details,
      page: () => const CountryDetailsView(),
      binding: BindingsBuilder(() => Get.lazyPut(() => CountryDetailsController())),
    ),
  ];
}
```

lib/widgets/country_map.dart 59 lines

```dart
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';

import '../core/config/app_config.dart';
import '../core/theme/app_colors.dart';

/// OpenStreetMap view centred on a location with a marker.
class CountryMap extends StatelessWidget {
  final double latitude;
  final double longitude;
  final double zoom;
  final bool interactive;

  const CountryMap({
    super.key,
    required this.latitude,
    required this.longitude,
    this.zoom = 4,
    this.interactive = true,
  });

  @override
  Widget build(BuildContext context) {
    final point = LatLng(latitude, longitude);

    return FlutterMap(
      options: MapOptions(
        initialCenter: point,
        initialZoom: zoom,
        interactionOptions: InteractionOptions(
          flags: interactive
              ? InteractiveFlag.all & ~InteractiveFlag.rotate
              : InteractiveFlag.none,
        ),
      ),
      children: [
        TileLayer(
          urlTemplate: AppConfig.mapTileUrl,
          userAgentPackageName: AppConfig.mapUserAgent,
        ),
```

```
        MarkerLayer(
          markers: [
            Marker(
              point: point,
              width: 48,
              height: 48,
              alignment: Alignment.topCenter,
              child: const Icon(Icons.location_on, size: 48, color: AppColors.favorite),
            ),
          ],
        ),
        const RichAttributionWidget(
          attributions: [TextSourceAttribution('OpenStreetMap contributors')],
        ),
      ],
    );
  }
}
```

lib/widgets/flag_image.dart 53 lines

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';

import '../core/theme/app_colors.dart';

/// Country flag loaded from the network and cached; falls back to the emoji flag.
class FlagImage extends StatelessWidget {
  final String url;
  final String emoji;
  final double width;
  final double height;
  final double radius;
  final BoxFit fit;

  const FlagImage({
    super.key,
    required this.url,
    required this.emoji,
    this.width = 64,
    this.height = 44,
    this.radius = 10,
    this.fit = BoxFit.cover,
  });

  @override
  Widget build(BuildContext context) {
    final fallback = Container(
      width: width,
      height: height,
      color: AppColors.background,
      alignment: Alignment.center,
      child: Text(emoji, style: TextStyle(fontSize: height * 0.6)),
    );

    return ClipRRect(
      borderRadius: BorderRadius.circular(radius),
      child: url.isEmpty
          ? fallback
          : CachedNetworkImage(
              imageUrl: url,
              width: width,
              height: height,
              fit: fit,
              placeholder: (_, _) => Container(
                width: width,
                height: height,
                color: AppColors.border,
              ),
              errorWidget: (_, _, _) => fallback,
            ),
    );
  }
}
```

lib/widgets/section_card.dart 50 lines

```dart
import 'package:flutter/material.dart';

import '../core/theme/app_colors.dart';
import '../core/theme/app_theme.dart';

/// White rounded card with an icon + title header, used on the details screen.
class SectionCard extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final Widget? trailing;

  const SectionCard({
    super.key,
    required this.title,
    required this.icon,
    required this.child,
    this.trailing,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(AppSpacing.md),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: BorderRadius.circular(AppSpacing.radius),
        border: Border.all(color: AppColors.border),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Icon(icon, size: 20, color: AppColors.primary),
              const SizedBox(width: AppSpacing.sm),
              Expanded(
                child: Text(title, style: Theme.of(context).textTheme.titleMedium),
              ),
              ?trailing,
            ],
          ),
          const SizedBox(height: AppSpacing.md),
          child,
        ],
      ),
    );
  }
}
```

lib/widgets/status_view.dart 78 lines

```dart
import 'package:flutter/material.dart';

import '../core/theme/app_colors.dart';
import '../core/theme/app_theme.dart';

/// Reusable centered message for empty, error and no-result states.
class StatusView extends StatelessWidget {
  final IconData icon;
  final String title;
  final String message;
  final String? actionLabel;
  final VoidCallback? onAction;
  final Color color;

  const StatusView({
    super.key,
    required this.icon,
    required this.title,
    required this.message,
    this.actionLabel,
    this.onAction,
    this.color = AppColors.primary,
  });

  /// Error state with a retry button.
```

```
  const StatusView.error({
    super.key,
    required this.message,
    this.onAction,
    this.title = 'Something went wrong',
    this.actionLabel = 'Try again',
  })  : icon = Icons.cloud_off_rounded,
        color = AppColors.error;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(AppSpacing.xl),
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 360),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Container(
                padding: const EdgeInsets.all(AppSpacing.lg),
                decoration: BoxDecoration(
                  color: color.withValues(alpha: 0.1),
                  shape: BoxShape.circle,
                ),
                child: Icon(icon, size: 44, color: color),
              ),
              const SizedBox(height: AppSpacing.lg),
              Text(title, style: textTheme.titleLarge, textAlign: TextAlign.center),
              const SizedBox(height: AppSpacing.sm),
              Text(
                message,
                style: textTheme.bodyMedium?.copyWith(color: AppColors.textSecondary),
                textAlign: TextAlign.center,
              ),
              if (actionLabel != null && onAction != null) ...[
                const SizedBox(height: AppSpacing.lg),
                FilledButton.icon(
                  onPressed: onAction,
                  icon: const Icon(Icons.refresh_rounded),
                  label: Text(actionLabel!),
                  style: FilledButton.styleFrom(minimumSize: const Size(160, 48)),
                ),
              ],
            ],
          ),
        ),
      ),
    );
  }
}
```

test/models_test.dart 112 lines

```
import 'package:country_wearther_task/core/network/app_exception.dart';
import 'package:country_wearther_task/core/utils/formatters.dart';
import 'package:country_wearther_task/core/utils/validators.dart';
import 'package:country_wearther_task/data/models/country_model.dart';
import 'package:country_wearther_task/data/models/weather_model.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  group('Country.fromJson', () {
    test('parses a REST Countries v5 record', () {
      final country = Country.fromJson({
        'names': {'common': 'Canada', 'official': 'Canada'},
        'codes': {'alpha_3': 'CAN'},
        'capitals': [
          {'name': 'Ottawa', 'coordinates': {'lat': 45.42, 'lng': -75.7}},
        ],
        'flag': {'emoji': '\u{1F1E8}\u{1F1E6}', 'url_png': 'https://flags.restcountries.com/v5/w640/ca.png'},
        'region': 'Americas',
        'subregion': 'North America',
        'population': 41798407,
        'currencies': [
          {'code': 'CAD', 'name': 'Canadian dollar', 'symbol': r'$'},
        ],
        'coordinates': {'lat': 60, 'lng': -95},
      });
```

```
26
27        expect(country.code, 'CAN');
28        expect(country.name, 'Canada');
29        expect(country.capital, 'Ottawa');
30        expect(country.region, 'Americas');
31        expect(country.population, 41798407);
32        expect(country.currencyLabel, r'Canadian dollar ($)');
33        expect(country.latitude, 60);
34        expect(country.longitude, -95);
35        expect(country.hasLocation, isTrue);
36      });
37
38      test('handles missing fields without crashing', () {
39        final country = Country.fromJson({});
40
41        expect(country.name, 'Unknown');
42        expect(country.capital, 'N/A');
43        expect(country.region, 'N/A');
44        expect(country.population, isNull);
45        expect(country.currencyLabel, 'N/A');
46        expect(country.hasLocation, isFalse);
47      });
48
49      test('falls back to capital coordinates', () {
50        final country = Country.fromJson({
51          'names': {'common': 'Test'},
52          'capitals': [
53            {'name': 'Cap', 'coordinates': {'lat': 1.5, 'lng': 2.5}},
54          ],
55        });
56
57        expect(country.latitude, 1.5);
58        expect(country.longitude, 2.5);
59      });
60    });
61
62    group('Weather.fromJson', () {
63      test('parses Open-Meteo current block', () {
64        final weather = Weather.fromJson({
65          'current': {
66            'temperature_2m': 26.7,
67            'apparent_temperature': 27.4,
68            'relative_humidity_2m': 47,
69            'wind_speed_10m': 5.2,
70            'weather_code': 0,
71            'is_day': 1,
72          },
73        });
74
75        expect(weather.temperature, 26.7);
76        expect(weather.humidity, 47);
77        expect(weather.windSpeed, 5.2);
78        expect(weather.condition, 'Clear sky');
79        expect(weather.isDay, isTrue);
80      });
81
82      test('throws AppException on invalid response', () {
83        expect(() => Weather.fromJson({'foo': 'bar'}), throwsA(isA<AppException>()));
84      });
85    });
86
87    group('Validators', () {
88      test('email', () {
89        expect(Validators.email(''), isNotNull);
90        expect(Validators.email('wrong'), isNotNull);
91        expect(Validators.email('user@example.com'), isNull);
92      });
93
94      test('password', () {
95        expect(Validators.password(''), isNotNull);
96        expect(Validators.password('123'), isNotNull);
97        expect(Validators.password('123456'), isNull);
98      });
99    });
100
101   group('Formatters', () {
102     test('groupedNumber', () {
103       expect(Formatters.groupedNumber(41798407), '41,798,407');
104       expect(Formatters.groupedNumber(null), 'N/A');
105     });
106
107     test('coordinates', () {
```

```
108         expect(Formatters.latitude(-12.5), '12.5000° S');
109         expect(Formatters.longitude(77), '77.0000° E');
110       });
111     });
112   }
```